Segmenting Bath Soap Households by Purchase Behaviour

ETW2500 Unsupervised Learning for Business, Assignment 3, Semester 2, 2026

Erwyna Soo Wen Xin

29 October 2026

Table of contents

1 Introduction 3
1.1 Business problem 3
1.2 Objectives 4
1.3 Approach 4
2 How to Reproduce This Report 4
3 Setup and Data 5
3.1 Packages, seed and theme 5
3.2 Reading the data 6
3.3 First look 7
4 Question 1, Part A: Thinking About Brand Loyalty 7
4.1 Loyalty of households X and Y (item 1) 8
4.2 Euclidean distances between X, Y and Z (item 2) 8
4.3 Using the nine brand shares unchanged (item 3) 10
5 Question 1, Part B: Creating MaxBrShare 12
5.1 Creating the variable (item 4) 12
5.2 Checking the variable (item 5) 12
5.3 Repeating the distance calculation (item 6) 13
5.4 Reading MaxBrShare with NoBrands (item 7) 14
6 Question 2(a): Choosing the Variables 14
6.1 The four ideas of purchase behaviour 14
6.2 Selected variables 15
6.3 Variables that are exact copies of each other 16
6.4 Why the others are excluded 18
6.5 What was considered for each idea 22
6.6 Checking the six for multicollinearity 23
6.7 Limits of this choice 25
7 Question 2(b): Preparing the Data 26
7.1 Step 1: Missing values hidden as zeros 26
7.2 Step 2: Skewness 29
7.3 Step 3: Standardisation 32

7.4 Step 4: Outliers 35
8 Question 2(c) and 2(d): Clustering the Households 37
8.1 Choosing the method 37
8.2 Choosing the number of groups 38
8.3 The final grouping 41
8.3.a Does the result depend on the method? 41
8.3.b How stable are the six segments? 44
8.4 Describing each group (2d) 47
9 Question 2(e): Profiling the Groups 51
9.1 Which three demographics, and why 51
9.2 Profile by group 53
9.2.a Socioeconomic class 54
9.2.b Household size 55
9.2.c Age of the homemaker 56
9.3 Are the differences real? 57
9.4 Summary profile of each segment 58
10 Question 3: Dimension Reduction with PCA 60
10.1 Is PCA worth doing on these variables? 60
10.2 Running the PCA 61
10.3 How many components to keep 62
10.4 Naming and describing the components 64
10.5 The households on the components 67
10.6 What PCA gives up 69
11 Conclusion and Recommendations 70
11.1 What we found 70
11.2 Recommendations for XYZ's clients 70
11.3 Limitations 73
11.4 Next steps 73
Appendix 73
Further checks 73
Question 1(b): a second method gives the same MaxBrShare 73
Question 1(b): how concentration is spread 74
Question 1(b): do the nine shares add up to 1? 75
Question 2(e): where do the differences come from? 75
Question 2(e): did we miss a better demographic? 77
Packages used 78
Session information 79
Bibliography 81

Group members (group leader first)

Name	Student ID
Erwyna Soo Wen Xin (Group Leader)	36555789
Taabish Farooq Bhat	35473932
Nandhini Hariharan	36296864

Name	Student ID
Riely Hariri Bin Rafliz	36273163
Naidu Shrrii Shivanie	35395346

1 Introduction

Executive summary

- **Six behavioural segments.** Ward's hierarchical clustering on six purchase variables (volume, frequency, two promotion shares, number of brands and `MaxBrShare`) splits the households into Small-brand regular-price buyers, Heavy multi-brand shoppers, Loyal regular-price buyers, Promo-6 buyers, Loyal promotion buyers and Light occasional buyers (sizes in Table 25).
- **Loyalty is measured as concentration, not brand.** Raw brand shares would group households by *which* brand they buy; the single variable `MaxBrShare` measures *how concentrated* their buying is, which is what loyalty means (Question 1).
- **Three dimensions name the segments.** PCA keeps three components (Table 30): shopping activity and brand variety, loyal bulk buying against promotion 6, and response to other promotions. The segments line up with these dimensions (Table 32), which agrees with the clustering but is not independent proof, since both use the same six variables.
- **Demographics barely separate the segments.** Socioeconomic class and household size differ only slightly and age not at all, so clients should target these segments with purchase data, using demographics only to fine-tune.
- **One approach per segment.** Rewards without discounts for the Loyal regular-price buyers, price promotions on the brand they already favour for the Loyal promotion buyers, brand-tied multi-packs for the Heavy multi-brand shoppers, well-timed promotion 6 offers for the Promo-6 buyers, trial and visibility for the Small-brand regular-price buyers, and the lowest priority for the Light occasional buyers. The main caution is that segment borders overlap, and some segments are much sharper than others (Section 8.3.b).

1.1 Business problem

XYZ is an Asian market research agency that tracks the purchase behaviour of household panels across more than 100 cities and towns. Its clients are advertising agencies, which use the data to advise on advertising and promotion, and consumer goods manufacturers, which use it to monitor market share. Until now XYZ has segmented markets by purchaser *demographics*. This report segments the bath soap market by purchase *behaviour* instead: how much a household buys (volume), how often it buys (frequency), how strongly it responds to discounts (promotion sensitivity) and how loyal it is to a brand.

Segmenting on behaviour matters commercially because a promotion aimed at the right group is cheaper than one aimed at everyone. If households that only buy on discount can be told apart from households that are loyal to one brand, a client can run several different promotions at different times of the year, and can design reward schemes that deepen brand

loyalty where it already exists. The demographics are then used afterwards to *profile* each behavioural segment, which shows who the segments are.

1.2 Objectives

This report answers three questions on a panel of households:

1. **Question 1, brand loyalty.** Why raw brand shares are a poor basis for measuring loyalty, and how to replace them with a single variable, `MaxBrShare`.
2. **Question 2, segmentation.** Select behavioural variables, prepare them, group the households with cluster analysis, describe each group and profile the groups with demographics.
3. **Question 3, dimension reduction.** Summarise the six segmentation variables into a smaller number of components with principal component analysis.

1.3 Approach

The analysis runs in one line of dependencies. The loyalty variable `MaxBrShare` (Question 1) feeds the choice of six behaviour variables (Question 2a). Those six are cleaned, corrected for skewness and standardised (Question 2b). The prepared data are then clustered and the groups described (Question 2c and 2d) and profiled with demographics (Question 2e). The same six prepared variables are also summarised by principal component analysis (Question 3).

2 How to Reproduce This Report

Every result in this report is calculated by the visible code from the two data files supplied. Follow these steps to rebuild it from the submitted project.

1. **Unzip the project and keep its layout.** The folder contains `ErwynaSooW.Rproj`, a `dataset/` folder (`bathsoap.csv` and `brand_example.csv`) and a `document/` folder (`ErwynaSooW.qmd`, its rendered HTML and PDF, the bibliography and the stylesheet). Do not move or rename any file.
2. **Install the software.** You need R and the Quarto CLI. RStudio and VS Code are alternative editors. The versions that produced this report are in Table 1. Package versions are listed in the appendix.

```
knitr::kable(tibble::tibble(
  Software = c("R", "Quarto CLI", "Platform"),
  Version  = c(as.character(getRversion()),
               trimws(system2("quarto", "--version", stdout = TRUE)),
               R.version$platform)
))
```

Table 1: Software versions used to render this report.

Software	Version
R	4.6.0
Quarto CLI	1.10.18
Platform	aarch64-apple-darwin23

3. **Open the project root.** In RStudio, open `ErwynaSooW.Rproj` first. In VS Code, use **File** > **Open Folder** to open the folder containing that file, then open a terminal in that folder. This gives `here()` the project root it uses to find `dataset/`. Do not move or rename the data files.

4. **Prepare the packages once.** In an R console, install the bootstrap and rendering packages with `install.packages(c("pacman", "knitr", "rmarkdown", "ragg"), repos = "https://cloud.r-project.org")`. The shared setup uses `pacman::p_load()` and `p_install()` to install any other missing package (`tidyverse`, `tidymodels`, `here`, `janitor`, `gt`, `scales`, `patchwork`, `factoextra`, `cluster`, `moments`, `mclust`), so the first render needs internet access and a writable R library. Figures use the `ragg` device.

5. **Render both required formats.** Open `document/ErwynaSooW.qmd` in RStudio and use the **Render** dropdown to select each format, or run these commands in a terminal at the project root:

```
quarto render document/ErwynaSooW.qmd --to html
quarto render document/ErwynaSooW.qmd --to typst
```

The outputs are `document/ErwynaSooW.html` (the main document) and `document/ErwynaSooW.pdf` (the attachment). The PDF uses Typst, so no LaTeX installation is needed. The single render command without `--to` also produces both formats in the Quarto version recorded above.

6. **Repeatable results.** The setup chunk sets `set.seed(55789)`, the last five digits of the group leader's student ID. Hierarchical clustering and PCA are not random, so the seed only matters for the simulated p-values and the k-means check, which uses random starts. Keeping the code, data, package versions, seed and order of random operations fixed makes the numerical results repeatable.

7. **If something fails.** If `quarto` is not recognised, install the Quarto CLI, add its `bin` folder to `PATH`, and reopen your editor or terminal. If a package fails to install, run `install.packages("name")` once and render again. If a data file cannot be found, check that you opened the `.Rproj` file and that the `dataset/` folder sits next to it.

3 Setup and Data

3.1 Packages, seed and theme

All packages are loaded in one chunk, before anything else runs. `pacman::p_load()` installs any package that is missing on the reader's machine, so the report renders without a separate installation step. There is no `install.packages()` call anywhere in this document. The analysis is written in R [1] with the tidyverse [2] for data handling and graphics and tidymodels [3], [4] for the preparation and PCA recipes. Clustering uses `cluster` [5] and `factoextra` [6], tables use `gt` [7], paths use `here` [8], cleaning uses `janitor` [9], combined figures use `patchwork` [10], the adjusted Rand index comes from `mclust` [11] and skewness from `moments` [12].

```
# pacman installs anything missing, then loads it
library(pacman)
p_load(tidyverse, tidymodels, here, janitor, gt, scales, patchwork,
       factoextra, cluster, moments)
# mclust is installed but not attached: its map() would hide purrr::map(),
# so it is called as mclust::adjustedRandIndex()
p_install(mclust, force = FALSE)

# The seed is the last five digits of the group leader's student ID.
# Clustering and any random split depend on it, so setting it once, here,
# makes every result in this report repeat exactly.
set.seed(55789)

# One flat purple and pink palette and one chart theme for the whole report,
so every
# figure matches the HTML styling (document/style.css uses the same colours).
# pal[1] is the main colour and pal[4] its opposite in the diverging scales.
pal    <- c("#8A77DD", "#F28DB5", "#7DB3E8", "#F1CF68", "#8FD3BD", "#C9A0E8")
pal_grey <- "#D9D3EA"   # neutral, for "not selected" bars
pal_ink  <- "#34304A"   # text and arrows
pal_lo   <- "#A9CDF0"   # light ends of the diverging scales, so dark text
pal_hi   <- "#E7A0C0"   # stays readable on top of every cell
theme_set(
  theme_minimal(base_size = 13) +
    theme(plot.title = element_text(face = "bold", colour = pal_ink),
          text = element_text(colour = pal_ink),
          axis.text = element_text(colour = "#6B6680"),
          panel.grid.minor = element_blank(),
          panel.grid.major = element_line(colour = "#ECE6F7"),
          legend.position = "bottom")
)
```

3.2 Reading the data

The file `bathsoap.csv` has one row per household. `brand_example.csv` is the three household toy example used in Question 1. Both are read with `here()`, which builds the path from the project root (the folder that holds `ErwynaSooW.Rproj`), so the paths work on any computer.

```
soap_raw      <- read_csv(here("dataset", "bathsoap.csv"),
                          show_col_types = FALSE)
brand_example <- read_csv(here("dataset", "brand_example.csv"),
                          show_col_types = FALSE)

# Column groups, named once so every section uses the same definitions.
# The eight named brands are used for loyalty; BrOthers pools many small brands
# and is deliberately left out of that list (see Question 1).
brand_cols <- c("Br57_144", "Br55", "Br272", "Br286", "Br24", "Br481",
```

```
                "Br352", "Br5")
demo_cols  <- c("SEC", "FEH", "MT", "SEX", "AGE", "EDU", "HS", "CHILD", "CS",
                "Affluence")
```

3.3 First look

```
# Households whose nine brand shares do not add up to 1 within 0.01.
# Defined once here so other sections can reuse it.
share_exceptions <- soap_raw |>
  mutate(share_total = rowSums(pick(all_of(brand_cols), BrOthers))) |>
  filter(abs(share_total - 1) >= 0.01) |>
  select(MemberID, share_total)

tibble(
  households        = nrow(soap_raw),
  columns           = ncol(soap_raw),
  unique_ids        = n_distinct(soap_raw$MemberID),
  missing_cells     = sum(is.na(soap_raw)),
  shares_off        = nrow(share_exceptions)
) |>
  gt() |>
  cols_label(households = "Households", columns = "Columns",
             unique_ids = "Unique IDs", missing_cells = "Empty cells (NA)",
             shares_off = "Brand shares not summing to 1 (within 0.01)")
```

Table 2: Basic checks on bathsoap.csv.

Households	Columns	Unique IDs	Empty cells (NA)	Brand shares not summing to 1 (within 0.01)
600	46	600	0	2

Table 2 shows that the data have 600 households and no empty cells, and all but 2 households have nine brand shares that add up to 1 within 0.01 (the exceptions are listed in the appendix). A clean-looking table does **not** mean there is no missing information. The data guide says a code of 0 means "not specified" for the demographic variables, so the missing values are hidden inside the numbers. Section 7 finds and handles them.

4 Question 1, Part A: Thinking About Brand Loyalty

Before any calculation, this part asks what brand loyalty actually is, and whether the nine brand share columns measure it. The answer decides how the loyalty variable is built in Section 5 and which variables enter the segmentation in Section 6.

4.1 Loyalty of households X and Y (item 1)

Household X buys only brand A and household Y buys only brand B. The two households are **equally loyal**. Each puts 100% of its purchases into a single brand, so neither splits its loyalty across alternatives.

Brand loyalty therefore depends on **how concentrated a household's purchases are**, not on **which brand** receives them. Two things carry that information:

- **Share:** the proportion of a household's volume that goes to its most purchased brand. A share of 1 is complete loyalty and a share of 0.5 is a household that divides itself between two brands.
- **Concentration:** how many brands the volume is spread over. The fewer the brands, and the more uneven the split, the more loyal the household.

The name of the brand is irrelevant. A household that gives 90% of its volume to brand A and a household that gives 90% to brand B are the same kind of customer, and a client would use the same retention promotion on both.

4.2 Euclidean distances between X, Y and Z (item 2)

Euclidean distance is the square root of the sum of squared differences across the variables. Here the variables are the two brand shares, so the distance between households i and j is $\sqrt{(A_i - A_j)^2 + (B_i - B_j)^2}$. A small distance means the two households look alike to a distance based method.

```
# dist() needs numbers only, so the Household names move into the row names.
# as.data.frame() first, because tibbles do not keep row names.
example_shares <- brand_example |>
  as.data.frame() |>
  column_to_rownames("Household")

# Euclidean distance between every pair of rows (the default method of dist())
example_dist <- dist(example_shares, method = "euclidean")

# Turn the dist object into a square table with the household names restored
as.matrix(example_dist) |>
  as_tibble(rownames = "Household") |>
  gt() |>
  fmt_number(columns = -Household, decimals = 3)
```

Table 3: Euclidean distance between households X, Y and Z.

Household	X	Y	Z
X	0.000	1.414	0.707
Y	1.414	0.000	0.707
Z	0.707	0.707	0.000

```
# One row per pair (each pair once), sorted from closest to furthest
example_pairs <- as.matrix(example_dist) |>
  as_tibble(rownames = "from") |>
  pivot_longer(-from, names_to = "to", values_to = "distance") |>
  filter(from < to) |>
  arrange(distance)

example_pairs |>
  mutate(pair = str_c(from, " and ", to)) |>
  select(pair, distance) |>
  gt() |>
  fmt_number(columns = distance, decimals = 3) |>
  cols_label(pair = "Pair of households", distance = "Euclidean distance")
```

Table 4: Each pair of toy households, from closest to furthest.

Pair of households	Euclidean distance
X and Z	0.707
Y and Z	0.707
X and Y	1.414

```
# Straight line segments between each pair, labelled with their distance
segment_tbl <- example_pairs |>
  left_join(rename(brand_example, from = Household, x1 = BrandA, y1 = BrandB),
            by = "from") |>
  left_join(rename(brand_example, to = Household, x2 = BrandA, y2 = BrandB),
            by = "to") |>
  # Label at the midpoint, moved off the line for the longest pair so it does
  # not cover Z
  mutate(label_x = (x1 + x2) / 2 + if_else(distance == max(distance), 0.18, 0),
         label_y = (y1 + y2) / 2 + if_else(distance == max(distance), 0.18, 0))

ggplot(brand_example, aes(BrandA, BrandB)) +
  geom_abline(intercept = 1, slope = -1, linetype = "dashed",
              colour = "grey60") +
  geom_segment(data = segment_tbl, aes(x = x1, y = y1, xend = x2, yend = y2),
               colour = pal[2], linewidth = 1) +
  geom_label(data = segment_tbl,
             aes(x = label_x, y = label_y, label = round(distance, 3)),
             size = 3.5, colour = pal_ink) +
  geom_point(size = 5, colour = pal[1]) +
  geom_text(aes(label = Household), nudge_x = 0.04, nudge_y = 0.05,
            fontface = "bold") +
  coord_equal(xlim = c(-0.1, 1.15), ylim = c(-0.1, 1.15)) +
```

```
  labs(title = "X and Y are furthest apart, yet equally loyal",
       x = "Share of volume in brand A", y = "Share of volume in brand B")
```

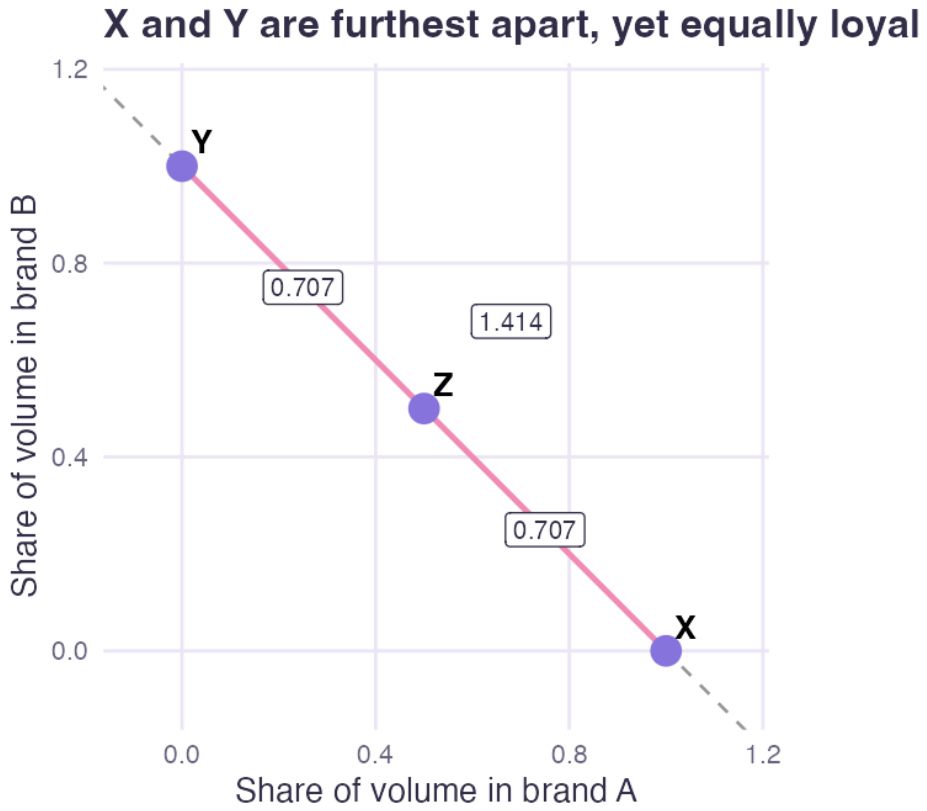


Figure 1: Households X, Y and Z by brand share. Every household whose two shares add up to 1 lies on the dashed line.

Table 3 gives the full distance matrix and Table 4 ranks the pairs. The closest pairs are (X, Z) and (Y, Z), tied at 0.707, while X and Y are the furthest apart at 1.414. Household Z splits its volume evenly between the two brands, so it sits halfway between X and Y and is the same distance from each.

This is not consistent with item 1. In item 1, X and Y are equally loyal, so they should be the most similar pair on loyalty. The distances say the opposite: X and Y are the most different pair, and each looks more like the half and half household Z. The distance is reading *which brand* was bought, and is blind to *how concentrated* the buying is.

The geometry shows why. Every household with two shares that add up to 1 lies on the dashed line in Figure 1, and distance along that line measures how far apart two households are in brand choice. A household with shares 0.9 and 0.1 would sit very close to X (a distance of 0.141) and far from Y (1.273), even though a 90% concentration is nearly as loyal as Y's 100%.

4.3 Using the nine brand shares unchanged (item 3)

Distance based methods such as cluster analysis group households that are close together, so they will inherit the problem in item 2. If the nine brand share variables (eight named brands and `BrOthers`) enter the analysis unchanged, three things go wrong.

1. **Households group by the brand they buy, not by how loyal they are.** Two households that each give all of their volume to a single brand have a squared distance of exactly 2 if the brands differ, and 0 if the brands are the same. So fully loyal households form one tight group per brand and sit far from each other, even though they behave identically.
2. **Loyalty gets far too much weight.** Nine of the columns would describe brand choice, so one idea (loyalty) would outvote volume, frequency and promotions, which would each be a single column.
3. **The nine shares are not independent.** They add up to 1 (for all but 2 households, see Section 5), so any one is almost fixed by the other eight. A distance method counts the same information nine times over.

The next chunk checks this on the real data, using only the households that put **all** of their volume into one named brand.

```
# The nine share columns: eight named brands plus the pooled BrOthers
nine_cols <- c(brand_cols, "BrOthers")

# Households that put all of their volume into one single named brand.
# max.col() finds, for each row, the column (brand) that holds the largest
# share.
loyal <- soap_raw |>
  # near(): safe comparison for decimals
  filter(if_any(all_of(brand_cols), \(x) near(x, 1))) |>
  mutate(brand = brand_cols[max.col(across(all_of(brand_cols)),
                                    ties.method = "first")])

# Pairwise Euclidean distance on the nine shares, kept as a square matrix
loyal_dist <- as.matrix(dist(select(loyal, all_of(nine_cols))))

# For each pair of households, were they loyal to the same brand?
loyal_pairs <- which(upper.tri(loyal_dist), arr.ind = TRUE) |>
  as_tibble() |>
  mutate(distance = loyal_dist[cbind(row, col)],
         same_brand = loyal$brand[row] == loyal$brand[col])

loyal_pairs |>
  count(same_brand, distance = round(distance, 3)) |>
  mutate(same_brand = if_else(same_brand, "Same brand", "Different brand"))
|>
  gt() |>
 cols_label(same_brand = "Loyal to", distance = "Distance on the nine shares",
            n = "Pairs of households")
```

Table 5: Fully loyal households: the distance on the nine shares depends only on the brand.

Loyal to	Distance on the nine shares	Pairs of households
Different brand	1.414	70
Same brand	0.000	35

Table 5 groups the pairs by distance. There are 15 households that are completely loyal to one named brand, which makes 105 pairs. Every pair that shares a brand has a distance of 0, because their rows are identical. Every pair with different brands has a distance of 1.414, the square root of 2, and these make up 67% of all pairs. Behaviour is identical in all 15 households, yet the unchanged shares push most of them as far apart as two single brand buyers can be. A clustering algorithm would split them by brand name, which a client cannot use. The brand shares have to be replaced by a variable that measures concentration and ignores identity, which is what `MaxBrShare` does in Section 5.

5 Question 1, Part B: Creating MaxBrShare

5.1 Creating the variable (item 4)

For household i, $\text{MaxBrShare}_i = \max_{b \in B} s_{ib}$, where B is the set of eight named brands and s_{ib} is brand b's share of that household's purchase volume. It measures how concentrated a household is on one named brand, whatever that brand is.

```
soap <- soap_raw |>
  # one household (row) at a time
  rowwise() |>
  # largest of the eight named shares
  mutate(MaxBrShare = max(c_across(all_of(brand_cols)))) |>
  # back to normal column operations
  ungroup()
```

The maximum is taken row by row because each household needs its own value. The maximum of a whole column would describe one brand across all households instead. `BrOthers` is excluded because it pools many small brands, so a high `BrOthers` share does not show loyalty to any single brand.

5.2 Checking the variable (item 5)

```
# Households with no volume in any of the eight named brands
zero_hh <- soap |> filter(near(MaxBrShare, 0))

# Five-number summary and mean, then the zero group as extra rows
enframe(summary(soap$MaxBrShare), name = "statistic", value = "value") |>
  mutate(value = sprintf("%.3f", as.numeric(value))) |>
  bind_rows(zero_hh |>
              count(NoBrands, name = "n") |>
              transmute(statistic = paste0("MaxBrShare = 0 and ",
                                           NoBrands, " brand(s): households"),
```

```
                      value = as.character(n))) |>
  gt() |>
  cols_label(statistic = "Statistic", value = "Value")
```

Table 6: Summary of MaxBrShare, and the number of brands bought by households with MaxBrShare = 0.

Statistic	Value
Min.	0.000
1st Qu.	0.134
Median	0.299
Mean	0.371
3rd Qu.	0.572
Max.	1.000
MaxBrShare = 0 and 1 brand(s): households	22

Table 6 checks the new variable. **22 households have `MaxBrShare` = 0, and every one of them has `NoBrands` = 1.** All values lie between 0 and 1, as a share must. These households buy a single brand, but that brand sits inside `BrOthers`. Once `BrOthers` is excluded, all eight named shares are zero, so their maximum is zero. A zero therefore means "no concentration on a *named* brand", not "no loyalty": these households are in fact completely loyal to one small brand.

5.3 Repeating the distance calculation (item 6)

```
example <- brand_example |>
  # largest share for each toy household
  mutate(MaxBrShare = pmax(BrandA, BrandB))

# Pairwise Euclidean distances for any set of columns, one row per pair
pair_dist <- function(cols) {
  m <- example |> select(all_of(cols)) |> as.matrix()
  rownames(m) <- example$Household
  as.matrix(dist(m)) |>
    as_tibble(rownames = "from") |>
    pivot_longer(-from, names_to = "to", values_to = "distance") |>
    filter(from < to)
}

dist_compare <- pair_dist(c("BrandA", "BrandB")) |>
  rename(original_shares = distance) |>
  left_join(pair_dist("MaxBrShare") |> rename(max_share = distance),
            by = c("from", "to"))
```

```
dist_compare |>
  gt() |>
  fmt_number(c(original_shares, max_share), decimals = 3) |>
  cols_label(from = "Household", to = "and",
             original_shares = "Original shares (Part A)",
             max_share = "MaxBrShare alone")
```

Table 7: Euclidean distances between X, Y and Z: original shares against MaxBrShare alone.

Household	and	Original shares (Part A)	MaxBrShare alone
X	Y	1.414	0.000
X	Z	0.707	0.500
Y	Z	0.707	0.500

Table 7 repeats the Part A distances. With the original shares, X and Y are the furthest apart (1.414) even though both are fully loyal. Using `MaxBrShare` alone, X and Y are 0 apart, identical, and each is 0.5 from Z, which splits its buying evenly. This fixes the problem from Section 4: distance now reflects *how concentrated* buying is, not *which brand* was bought.

5.4 Reading MaxBrShare with NoBrands (item 7)

`MaxBrShare` shows how concentrated a household is on its favourite named brand, while `NoBrands` shows how many brands it spreads across, so only together do they separate truly disloyal households from the 22 single-brand households whose `MaxBrShare` is zero only because their brand is in `BrOthers`.

Further checks on `MaxBrShare` (a second way to compute it, its distribution, and the households whose shares do not sum to 1) are in the Further checks part of the appendix.

6 Question 2(a): Choosing the Variables

The segmentation should group households by **how they buy bath soap**, so only variables that describe purchase behaviour can go in. The Key Problem names four ideas: **volume**, **frequency**, **susceptibility to discounts** and **brand loyalty**. Question 3 summarises "the six variables used in Question 2", so exactly six variables are chosen: one or two for each idea, with no two that say the same thing.

6.1 The four ideas of purchase behaviour

Idea	Variable(s) chosen	Why this one
Volume	`TotalVol`	The amount of soap bought in the period. It is the plain measure of how much a household buys.
Frequency	`NoTrans`	The number of shopping trips on which soap was

Idea	Variable(s) chosen	Why this one
		bought, so it is how often a household buys.
Susceptibility to discounts	`PurPromo6`, `PurOtherPromo`	The share of volume bought under promotion code 6 and under any other promotion. They are the two parts of discount buying that carry separate information.
Brand loyalty	`MaxBrShare`, `NoBrands`	The brief names three loyalty perspectives: the number of brands bought, how often a household switches brand, and the share of spending on one brand. `NoBrands` is the first and `MaxBrShare` (built in Section 5) is the third. Together they separate loyal households from those who only look disloyal. The switching perspective is discussed below.

6.2 Selected variables

```
# The six variables used for clustering (Question 2) and PCA (Question 3).
# The object name cluster_vars is used by sections 7, 8 and 10, so it must not
# change.
cluster_vars <- c("TotalVol",       # volume: total amount of soap bought
                  "NoTrans",        # frequency: number of purchase trips
                 # discounts: share of volume bought under promotion code 6
                  "PurPromo6",
                # discounts: share bought under other promotions (PurNoPromo
                  # is the leftover)
                  "PurOtherPromo",
                  "NoBrands",       # loyalty: how many brands are bought
                  # loyalty: concentration on one named brand (Question 1)
                  "MaxBrShare")

# Quick look at the six in their original units
soap |>
  select(all_of(cluster_vars)) |>
  summarise(across(everything(), list(mean = mean, sd = sd, min = min,
                                      max = max))) |>
```

```
  pivot_longer(everything(), names_to = c("variable", ".value"),
               # safe for names that contain "_"
               names_pattern = "(.+)_(mean|sd|min|max)") |>
  gt() |>
  fmt_number(columns = c(mean, sd, min, max), decimals = 2) |>
 cols_label(variable = "Variable", mean = "Mean", sd = "SD", min = "Minimum",
            max = "Maximum")
```

Table 8: The six clustering variables in original units.

Variable	Mean	SD	Minimum	Maximum
TotalVol	11,914.77	7,770.37	150.00	50,895.00
NoTrans	31.15	17.43	1.00	138.00
PurPromo6	0.05	0.09	0.00	0.67
PurOtherPromo	0.03	0.07	0.00	1.00
NoBrands	3.64	1.58	1.00	9.00
MaxBrShare	0.37	0.29	0.00	1.00

Table 8 summarises the six. They cover all four ideas. Volume and frequency have one variable each. Discounts and loyalty have two each, because in each case the two variables measure different things (the evidence is in Table 11 and Figure 2).

6.3 Variables that are exact copies of each other

Several columns are calculated from other columns, so they add nothing new and would count the same behaviour twice in a distance. The chunk below tests each suspected relationship on the data instead of assuming it.

```
# Each row of this table is one suspected relationship, written as a function of
# the data
redundancy_checks <- tribble(
  ~relationship,                                  ~expected, ~check,
  "PurNoPromo + PurPromo6 + PurOtherPromo", 1,
    \(d) d$PurNoPromo + d$PurPromo6 + d$PurOtherPromo,
  "PrCat1 + PrCat2 + PrCat3 + PrCat4",      1,
    \(d) d$PrCat1 + d$PrCat2 + d$PrCat3 + d$PrCat4,
  "PropCat5 + ... + PropCat15",             1,
    \(d) rowSums(d[paste0("PropCat", 5:15)]),
  "Value / (TotalVol x AvgPrice / 100)",    1,
    \(d) d$Value / (d$TotalVol * d$AvgPrice / 100),
  "VolPerTrans / (TotalVol / NoTrans)",     1,
    \(d) d$VolPerTrans / (d$TotalVol / d$NoTrans),
  "TransPerRun / (NoTrans / BrandRuns)",    1,
    \(d) d$TransPerRun / (d$NoTrans / d$BrandRuns)
)
```

```
# Run every check on all households and summarise how far from the expected
# value it strays
redundancy_checks |>
  mutate(result = map(check, \(f) f(soap)),
         smallest = map_dbl(result, min),
         largest  = map_dbl(result, max)) |>
  select(relationship, expected, smallest, largest) |>
  gt() |>
  fmt_number(columns = c(smallest, largest), decimals = 4) |>
  cols_label(relationship = "Relationship", expected = "Expected",
             smallest = "Smallest result", largest = "Largest result")
```

Table 9: Columns that are exact arithmetic of other columns.

Relationship	Expected	Smallest result	Largest result
PurNoPromo + PurPromo6 + PurOtherPromo	1	1.0000	1.0000
PrCat1 + PrCat2 + PrCat3 + PrCat4	1	1.0000	1.0000
PropCat5 + ... + PropCat15	1	1.0000	1.0000
Value / (TotalVol x AvgPrice / 100)	1	0.9991	1.0008
VolPerTrans / (TotalVol / NoTrans)	1	1.0000	1.0000
TransPerRun / (NoTrans / BrandRuns)	1	0.9964	1.0044

In every row the smallest and largest results sit on the expected value, to rounding error, for all 600 households. This leads to four decisions:

- **The three promotion shares add up to 1**, so any one is fixed by the other two. Only two can be used, because all three would make the variables perfectly collinear (one is a copy of the other two combined). `PurNoPromo` is the one dropped, since it is the leftover.
- **`Value` is `TotalVol` times `AvgPrice`** (divided by 100), so `Value`, `TotalVol` and `AvgPrice` describe one thing in three forms. Only `TotalVol` is kept, because volume is the behaviour and `AvgPrice` and `Value` mix in the price of the brands chosen.
- **`VolPerTrans` is `TotalVol` divided by `NoTrans`**, so it is built from two variables that are already in the six.
- **`TransPerRun` is `NoTrans` divided by `BrandRuns`**, so it is a ratio of a kept variable and another that is dropped below.

6.4 Why the others are excluded

```
# One row per column. The reason is written once per column (shared wording for
# groups).
demo_reason  <- str_c("Describes who the household is, not how it buys. ",
                      "Held back to profile the groups in Question 2(e).")
brand_reason <- str_c("The brief lists the brand shares as purchase ",
                      "behaviour, but each one measures which brand was ",
                     "bought, not how loyal the household is (Question 1 ",
                     "Part A), and the nine shares sum to 1. Replaced by ",
                      "MaxBrShare.")
prcat_reason <- str_c("Basis of purchase in the spec (a price category),
not ",
                         "purchase behaviour. The four shares sum to 1, so
they ",
                      "are redundant with each other.")
prop_reason  <- str_c("Basis of purchase in the spec (a selling proposition),
",
                       "not purchase behaviour. The eleven shares sum to 1,
so ",
                      "they are redundant with each other.")

# Helper: correlation of two columns, as text, so no number in a reason
is typed
# by hand
cor_text <- \(a, b) sprintf("%.2f", cor(soap[[a]], soap[[b]]))

variable_table <- bind_rows(
  tibble(variable = "MemberID", group = "Identifier", decision = "Dropped",
         reason = "A label for the household. It carries no behaviour."),
  tibble(variable = demo_cols, group = "Demographic", decision = "Dropped",
         reason = demo_reason),
  tribble(
    ~variable,       ~group,        ~decision, ~reason,
    "NoBrands", "Loyalty", "Kept",
    str_c("Breadth of brands bought. Read with MaxBrShare it tells loyal ",
          "buyers from those who only look disloyal (Question 1 item 7)."),
    "BrandRuns", "Loyalty", "Dropped",
   str_c("Counts runs of consecutive purchases of one brand, the switching ",
           "perspective of loyalty named in the brief. Left out because it
is ",
          "strongly correlated with NoTrans (",
          cor_text("BrandRuns", "NoTrans"), ") and NoBrands (",
          cor_text("BrandRuns", "NoBrands"),
          "), so it would count frequency and breadth a second time."),
    "TotalVol", "Volume", "Kept",
    "Total volume bought. The direct measure of how much a household buys.",
```

```
    "NoTrans", "Frequency", "Kept",
    str_c("Number of purchase trips. The direct measure of how often a ",
          "household buys."),
    "Value", "Volume", "Dropped",
    str_c("Equals TotalVol times AvgPrice / 100, so it repeats volume and ",
          "adds the price of the brand chosen (correlation ",
          cor_text("Value", "TotalVol"), " with TotalVol)."),
    "TransPerRun", "Loyalty", "Dropped",
     str_c("Equals NoTrans / BrandRuns, the average length of a brand run,
so ",
          "it is the other switching measure. Left out because it is built ",
          "from NoTrans, a kept variable (see the limits below); it also ",
          "overlaps with MaxBrShare (correlation ",
          cor_text("TransPerRun", "MaxBrShare"), ")."),
    "VolPerTrans", "Volume", "Dropped",
   str_c("Equals TotalVol / NoTrans, built from two variables already kept ",
          "(correlation ", cor_text("VolPerTrans", "TotalVol"),
          " with TotalVol)."),
    "AvgPrice", "Price", "Dropped",
    str_c("The brief lists it as a purchase summary, but it measures the ",
        "price level rather than one of the four ideas (volume, frequency, ",
         "discounts, loyalty). It depends on the brand and pack chosen and ",
          "it is the price part of Value."),
    "PurNoPromo", "Discounts", "Dropped",
    str_c("The leftover of the two promotion shares (all three sum to 1), ",
          "correlation ", cor_text("PurNoPromo", "PurPromo6"),
          " with PurPromo6. Using it too would add a redundant column."),
    "PurPromo6", "Discounts", "Kept",
   str_c("Share of volume under promotion code 6. Households differ widely ",
          "on it, and it is almost unrelated to PurOtherPromo."),
    "PurOtherPromo", "Discounts", "Kept",
    str_c("Share of volume under other promotions. A separate kind of ",
          "discount buying from PurPromo6.")
  ),
  tibble(variable = brand_cols, group = "Brand share", decision = "Dropped",
         reason = brand_reason),
  tribble(
    ~variable,       ~group,        ~decision, ~reason,
    "BrOthers", "Brand share", "Dropped",
     str_c("Pools many small brands, so it cannot identify loyalty to any
one ",
          "brand (Question 1 Part B). Included in the nine shares that ",
          "MaxBrShare replaces."),
    "MaxBrShare", "Loyalty", "Kept",
   str_c("The largest share of volume in one named brand, built in Question ",
          "1. It measures concentration and ignores which brand was bought.")
  ),
```

```
  tibble(variable = str_c("PrCat", 1:4), group = "Price category",
         decision = "Dropped", reason = prcat_reason),
  tibble(variable = str_c("PropCat", 5:15), group = "Proposition",
         decision = "Dropped", reason = prop_reason)
)

# Safety checks: every column of the data appears once, and the kept rows are
# exactly cluster_vars
stopifnot(setequal(variable_table$variable, names(soap)),
          !anyDuplicated(variable_table$variable),
        setequal(variable_table$variable[variable_table$decision == "Kept"],
                 cluster_vars))

# Show the table in the order of the data file
variable_table |>
  mutate(order = match(variable, names(soap))) |>
  arrange(order) |>
  select(variable, group, decision, reason) |>
  gt() |>
  cols_label(variable = "Variable", group = "Group", decision = "Decision",
             reason = "Reason") |>
  tab_style(style = cell_text(weight = "bold"),
            locations = cells_body(columns = decision,
                                   rows = decision == "Kept"))
```

Table 10: Every column in the data: kept or dropped, and why.

Variable	Group	Decision	Reason
MemberID	Identifier	Dropped	A label for the household. It carries no behaviour.
SEC	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
FEH	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
MT	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
SEX	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
AGE	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
EDU	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
HS	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
CHILD	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
CS	Demographic	Dropped	Describes who the household is, not how it buys. Held back to profile the groups in Question 2(e).
[illegible]	[illegible]	[illegible]	Counts runs of consecutive [illegible] Decreases who the household [illegible] count frequency and breadth a [illegible] second time.

Table 10 covers all 47 columns (the 46 in `bathsoap.csv` plus `MaxBrShare`): 6 are kept and 41 are dropped. The dropped columns fall into four kinds:

1. **Demographics** (10 columns). They describe the household, not its buying, so putting them in would group households by who they are. They are kept for profiling the segments in Section 9, where they tell a client who each segment is.
2. **Brand shares** (nine columns). They measure which brand was bought, which Section 4 showed would split fully loyal households by brand name. `MaxBrShare` replaces them.
3. **Basis of purchase** (`PrCat1` to `PrCat4` and `PropCat5` to `PropCat15`, 15 columns). The spec labels these as the basis of purchase and not as purchase behaviour. Each group also sums to 1.
4. **Columns that repeat the six** (`Value`, `AvgPrice`, `VolPerTrans`, `TransPerRun`, `BrandRuns`, `PurNoPromo`). They are either arithmetic on other columns or closely related to a kept variable, so they repeat information (see Table 9 and Table 12).

6.5 What was considered for each idea

Idea	Winner	Considered and rejected	Why the winner is better
Volume	`TotalVol`	`Value`, `VolPerTrans`	`Value` is volume multiplied by price, so it mixes behaviour with the cost of the brand. `VolPerTrans` is built from `TotalVol` and `NoTrans`.
Frequency	`NoTrans`	`BrandRuns`	`NoTrans` counts every purchase transaction. `BrandRuns` counts runs of the same brand, so it tracks switching more than how often a household buys.
Discounts	`PurPromo6` and `PurOtherPromo`	`PurNoPromo`, one combined promotion share	`PurNoPromo` is redundant. A single combined share (1 minus `PurNoPromo`) would hide that the two promotion types are almost uncorrelated.
Loyalty	`MaxBrShare` and `NoBrands`	The nine brand shares, `BrOthers`,	The nine shares measure brand iden-

Idea	Winner	Considered and rejected	Why the winner is better
		`BrandRuns`, `TransPerRun`	tity. `MaxBrShare` measures concentration, and `NoBrands` covers the households that buy one small brand and have `MaxBrShare` of 0. `BrandRuns` and `TransPerRun` measure switching, but they overlap with `NoTrans`, `NoBrands` and `MaxBrShare` (see Table 12), so they would count the same behaviour twice.

6.6 Checking the six for multicollinearity

Multicollinearity means two variables move together so closely that they carry almost the same information. If it were present, one idea would be counted twice in every distance. The correlation heatmap checks the six before they go on.

```
# Correlation matrix of the six, reshaped into one row per pair of variables
cor_pairs <- soap |>
  select(all_of(cluster_vars)) |>
  cor() |>
  as_tibble(rownames = "var1") |>
  pivot_longer(-var1, names_to = "var2", values_to = "r") |>
  mutate(across(c(var1, var2), \(x) factor(x, levels = cluster_vars)))

ggplot(cor_pairs, aes(var1, var2, fill = r)) +
  geom_tile(colour = "white") +
   geom_text(aes(label = sprintf("%.2f", r), colour = abs(r) > 0.6), size
= 3.6,
            show.legend = FALSE) +
  scale_colour_manual(values = c("TRUE" = pal_ink, "FALSE" = pal_ink)) +
  scale_fill_gradient2(low = pal_lo, mid = "white", high = pal_hi,
                       limits = c(-1, 1), name = "Correlation") +
  scale_y_discrete(limits = rev(cluster_vars)) +
   labs(title = "No pair of the six variables is strongly correlated", x
= NULL,
```

```
       y = NULL) +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

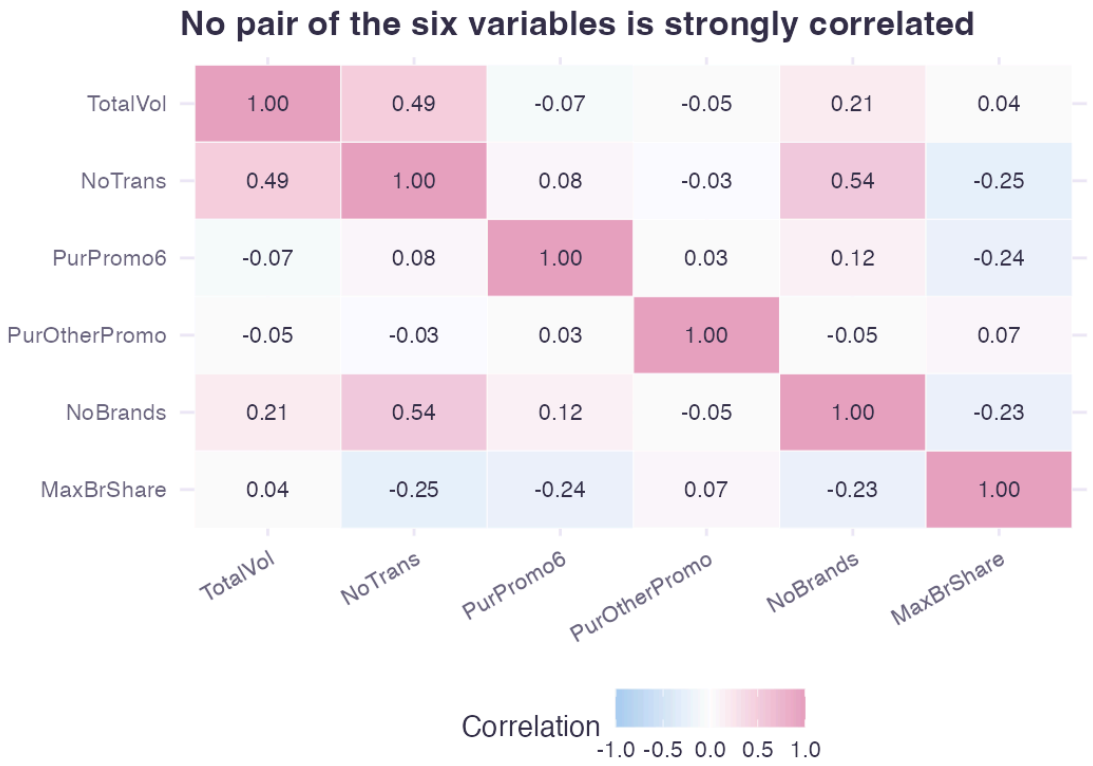


Figure 2: Correlation between the six chosen variables, in original units.

```
# Each pair once, ranked by the strength of the correlation (sign ignored)
strongest_pairs <- cor_pairs |>
  filter(as.integer(var1) < as.integer(var2)) |>
  arrange(desc(abs(r))) |>
  slice_head(n = 3)

strongest_pairs |>
  gt() |>
  fmt_number(columns = r, decimals = 2) |>
  cols_label(var1 = "Variable 1", var2 = "Variable 2", r = "Correlation")
```

Table 11: The three strongest correlations among the six chosen variables.

Variable 1	Variable 2	Correlation
NoTrans	NoBrands	0.54
TotalVol	NoTrans	0.49
NoTrans	MaxBrShare	–0.25

The strongest correlation is 0.54 between NoTrans and NoBrands, and **no pairs are above 0.7**. Households that shop more often tend to try more brands, which is expected and not close to a duplicate. The two promotion shares have a correlation of only 0.03, so the two kinds of discount buying are different behaviours and both are worth keeping. `MaxBrShare` and `NoBrands` have a correlation of −0.23, so the two loyalty variables overlap only a little.

```
# Candidates that were considered and dropped, against the six that were kept
dropped_candidates <- c("Value", "VolPerTrans", "BrandRuns", "TransPerRun",
                        "AvgPrice", "PurNoPromo")

soap |>
  select(all_of(dropped_candidates), all_of(cluster_vars)) |>
  cor() |>
  as_tibble(rownames = "candidate") |>
  filter(candidate %in% dropped_candidates) |>
  select(candidate, all_of(cluster_vars)) |>
  pivot_longer(all_of(cluster_vars), names_to = "closest", values_to = "r")
|>
  slice_max(abs(r), n = 1, by = candidate) |>
  arrange(desc(abs(r))) |>
  gt() |>
  fmt_number(columns = r, decimals = 2) |>
  cols_label(candidate = "Dropped candidate",
             closest = "Closest chosen variable", r = "Correlation")
```

Table 12: How strongly each dropped candidate is correlated with its closest chosen variable.

Dropped candidate	Closest chosen variable	Correlation
Value	TotalVol	0.88
PurNoPromo	PurPromo6	−0.80
BrandRuns	NoTrans	0.77
VolPerTrans	TotalVol	0.62
TransPerRun	MaxBrShare	0.42
AvgPrice	MaxBrShare	−0.27

Most dropped candidates have a strong link to one of the six (`Value`, `BrandRuns`, `PurNoPromo`), or are exact arithmetic of other columns (`VolPerTrans` from two kept variables, `TransPerRun` from a kept variable and `BrandRuns`), so dropping them loses little. `AvgPrice` is the weakest case for dropping, since its correlation with the six is modest. It was dropped because the price paid per unit reflects the brand and pack size chosen rather than how the household buys, and the Key Problem names volume, frequency, discounts and loyalty and not price.

6.7 Limits of this choice

- **The ideas are weighted unevenly.** Discounts and loyalty each have two variables and volume and frequency have one, so the segmentation gives loyalty and discounts more say

in the distances. The data support this (each pair has a low correlation inside it), but a different team could weight the four ideas equally.

- **The correlations are on the original units.** Several of the six are skewed, so the preparation step in Section 7 transforms and standardises them before clustering.
- **Switching between brands is not measured directly.** The brief names it as one of three loyalty perspectives. One rule was used for both switching measures: a column that is built from a variable already kept is dropped, whatever its correlation. `BrandRuns` counts runs inside the `NoTrans` purchase trips, and `TransPerRun` is `NoTrans` divided by `BrandRuns`, so both bring frequency back in a second time. That is why `TransPerRun` is dropped even though its correlation with `NoTrans` is only 0.02, against 0.77 for `BrandRuns`. Switching is still reflected indirectly through `NoBrands` and `MaxBrShare`, and Question 3 fixes the count at six, so adding one would mean removing another.
- **`MaxBrShare` of 0** applies to 22 households that buy only a brand outside the eight named. They are not disloyal, which is why `NoBrands` is in the six.

7 Question 2(b): Preparing the Data

Cluster analysis groups households by distance, so the data must be ready for distance calculations. Three things can distort a distance: values hiding as real numbers (missing data coded as 0), a few huge values that stretch one variable (skewness), and variables measured on very different scales (standardisation). They are handled in that order, because each step depends on the one before.

7.1 Step 1: Missing values hidden as zeros

The data guide gives two rules: for every demographic variable except `Affluence`, a value of 0 means "not specified", and `Affluence` is missing only when `HS` (household size) is 0. A household cannot have a size of 0, so a 0 there is also missing. Table 13 shows how many zeros each demographic variable holds before any change.

```
soap |>
  summarise(across(all_of(demo_cols), \(x) sum(x == 0))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "zeros") |>
  gt() |>
 cols_label(variable = "Demographic variable", zeros = "Households with a 0")
```

Table 13: Zeros in the demographic variables (before cleaning).

Demographic variable	Households with a 0
SEC	0
FEH	69
MT	69
SEX	68
AGE	0
EDU	73
HS	68
CHILD	0
CS	99
Affluence	69

`CHILD` has no zeros, yet the spec lists it as a demographic. Its categories run from 1 to 5, so Table 14 checks whether category 5 behaves like a "not specified" code.

```
soap |>
  count(`HS is 0` = HS == 0, CHILD) |>
  gt()
```

Table 14: CHILD category against HS = 0.

HS is 0	CHILD	n
FALSE	1	59
FALSE	2	145
FALSE	3	61
FALSE	4	267
TRUE	5	68

```
child5_all_hs0 <- soap |> filter(CHILD == 5) |> summarise(all(HS == 0)) |>
  pull()
child5_n       <- sum(soap$CHILD == 5)
hs0_n          <- sum(soap$HS == 0)
```

`CHILD = 5` appears in 68 households and **every one of them** has `HS = 0` (TRUE), while `HS = 0` is true for 68 households. The two flags describe the same households, so category 5 is treated as an undocumented "not specified" code for `CHILD`. This is an assumption, because the data guide does not say so, and it is recorded here so a reader can challenge it.

The next chunk applies the rules. The order matters: `Affluence` is checked against the *original* `HS` before `HS` itself has its zeros turned into `NA`.

```
soap_clean <- soap |>
  mutate(
    # Affluence 0 is missing only when HS is 0. This must run BEFORE HS is
    # changed.
    Affluence = replace(Affluence, Affluence == 0 & HS == 0, NA),
    # CHILD 5 is the undocumented 'not specified' code shown above.
    CHILD     = replace(CHILD, CHILD == 5, NA),
    # For the other demographics a 0 means 'not specified' (data guide).
    across(c(SEC, FEH, MT, SEX, AGE, EDU, HS, CHILD, CS), \(x) na_if(x, 0))
  )

missing_tbl <- soap_clean |>
  summarise(across(all_of(demo_cols), \(x) sum(is.na(x)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "n_missing")
|>
  mutate(percent = n_missing / nrow(soap_clean)) |>
  filter(n_missing > 0)

missing_tbl |>
  gt() |>
  cols_label(variable = "Demographic variable",
             n_missing = "Missing households",
             percent = "Share of households") |>
  fmt_percent(percent, decimals = 1)
```

Table 15: Missing demographic information after recoding zeros to NA.

Demographic variable	Missing households	Share of households
FEH	69	11.5%
MT	69	11.5%
SEX	68	11.3%
EDU	73	12.2%
HS	68	11.3%
CHILD	68	11.3%
CS	99	16.5%
Affluence	68	11.3%

```
n_any_missing <- soap_clean |> mutate(m = if_any(all_of(demo_cols), is.na))
|>
  summarise(sum(m)) |> pull()
```

```
n_cs_extra    <- sum(is.na(soap_clean$CS) & !is.na(soap_clean$HS))
n_purchase_na <- soap_clean |> select(-all_of(demo_cols), -MemberID) |>
  is.na() |> sum()

# The HS = 0 block: which demographics do they still have, and how much
do they
# buy?
hs0_block <- soap_clean |> filter(is.na(HS))
hs0_kept  <- demo_cols[map_lgl(demo_cols, \(v) !anyNA(hs0_block[[v]]))]
hs0_lost  <- setdiff(demo_cols, c(hs0_kept, "HS", "CHILD"))
hs0_all_lost <- all(map_lgl(hs0_lost, \(v) all(is.na(hs0_block[[v]]))))
vol_hs0   <- mean(hs0_block$TotalVol)
vol_other <- mean(soap_clean$TotalVol[!is.na(soap_clean$HS)])
```

Table 15 lists what is missing after the recode. In total 105 households (17.5%) have at least one demographic missing. Most of them are one block of 68 households with no household size. These households still have SEC and AGE recorded, but FEH, MT, SEX, EDU, CS and Affluence are all missing and `CHILD` is the code 5, so they lack the household composition, education and television details rather than every demographic. They also buy far less soap: 3,706 units on average, against 12,964 for the households with a recorded size. On top of this block, 31 further households have an unspecified `CS` (television availability). The purchase variables used for clustering have 0 missing values.

Decision. The segmentation uses purchase variables only, and those are complete, so no household is dropped and nothing is imputed for the clustering. Imputing household size, education and the other missing details for these 68 households would invent data. The demographics keep their `NA` and are used only to *profile* the groups in Question 2(e), where each variable is summarised over the households that have a value for it.

7.2 Step 2: Skewness

Skewness measures how lopsided a distribution is. In cluster analysis a few very large values pull the mean and stretch the distance on that variable, so the groups can end up being decided by the extreme households rather than the typical ones. The six selected variables are TotalVol, NoTrans, PurPromo6, PurOtherPromo, NoBrands, MaxBrShare.

```
# moments::skewness() is the third standardised moment.
# 0 is symmetric, positive means a long right tail.

soap_cluster_raw <- soap_clean |> select(MemberID, all_of(cluster_vars))

skew_before <- soap_cluster_raw |>
  summarise(across(all_of(cluster_vars), skewness)) |>
 pivot_longer(everything(), names_to = "variable", values_to = "skew_before")

soap_cluster_raw |>
  pivot_longer(all_of(cluster_vars), names_to = "variable",
               values_to = "value") |>
```

```
  ggplot(aes(value)) +
  geom_histogram(bins = 30, fill = pal[1], colour = "white") +
  facet_wrap(~variable, scales = "free") +
  # fewer, shorter axis labels so TotalVol's large values do not overlap
  scale_x_continuous(breaks = breaks_extended(n = 3),
                     labels = label_number(scale_cut = cut_short_scale())) +
  labs(x = NULL, y = "Households")
```

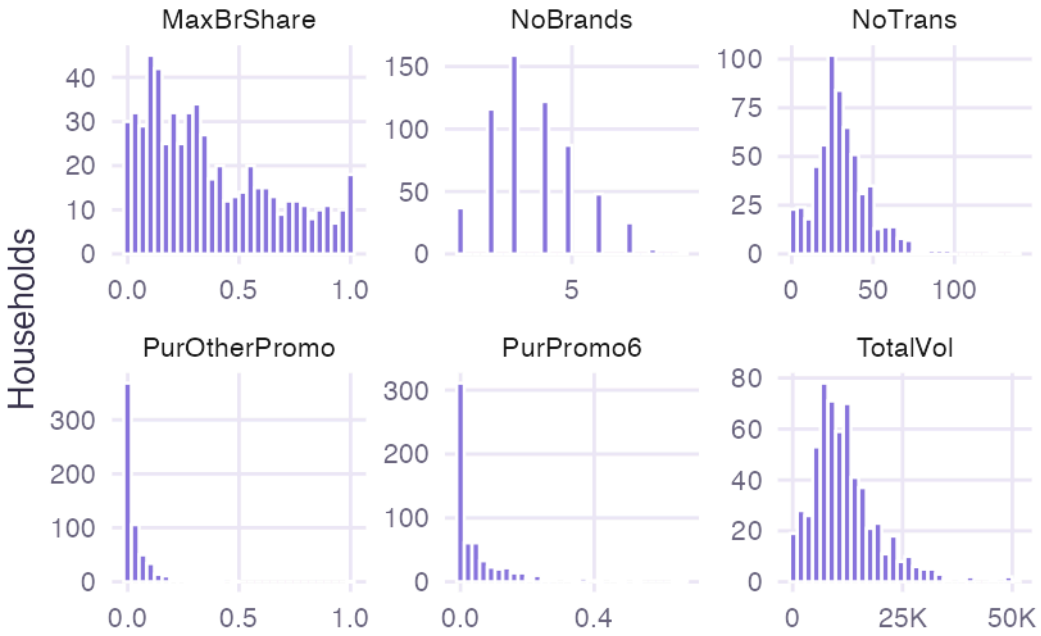


Figure 3: The six clustering variables before any transformation.

```
# Common rule of thumb: |skewness| above 1 is strongly skewed.
skewed_vars <- skew_before |> filter(abs(skew_before) > 1) |> pull(variable)

# Share of households with an exact zero in each variable. Promotion shares are
# often zero for households that never buy on that promotion (zero inflation).
zero_share <- soap_cluster_raw |>
  summarise(across(all_of(cluster_vars), \(x) mean(x == 0))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "zero_share")

# Rule used here: skewed AND at least 25% exact zeros -> square root;
# skewed with few zeros -> Yeo-Johnson; not strongly skewed -> leave alone.
sqrt_vars <- zero_share |> filter(variable %in% skewed_vars,
                                  zero_share >= 0.25) |> pull(variable)
yj_vars   <- setdiff(skewed_vars, sqrt_vars)

skew_before |>
  left_join(zero_share, by = "variable") |>
  mutate(method = case_when(variable %in% sqrt_vars ~ "Square root",
```

```
                          variable %in% yj_vars   ~ "Yeo-Johnson",
                          TRUE                    ~ "None needed")) |>
  gt() |>
  fmt_number(skew_before, decimals = 2) |>
  fmt_percent(zero_share, decimals = 0) |>
  cols_label(variable = "Variable", skew_before = "Skewness",
             zero_share = "Exact zeros", method = "Treatment")
```

Table 16: Skewness before transformation and the treatment chosen.

Variable	Skewness	Exact zeros	Treatment
TotalVol	1.45	0%	Yeo-Johnson
NoTrans	1.26	0%	Yeo-Johnson
PurPromo6	2.81	50%	Square root
PurOtherPromo	5.97	56%	Square root
NoBrands	0.52	0%	None needed
MaxBrShare	0.66	4%	None needed

```
# A trial recipe with Yeo-Johnson on the zero-heavy shares
yj_rec <- recipe(~ PurPromo6 + PurOtherPromo, data = soap_cluster_raw) |>
  step_YeoJohnson(all_predictors()) |>
  prep()
yj_kept <- tidy(yj_rec, number = 1)   # one row per variable that got a power

# The same search recipes runs (its internal objective, limits -5 to 5)
yj_best <- function(x) {
  optimize(recipes:::yj_obj, interval = c(-5, 5), maximum = TRUE,
           dat = x, ind_neg = list(is = which(x < 0), not = which(x >= 0)),
           const = sum(sign(x) * log(abs(x) + 1)), tol = 1e-4)$maximum
}

yj_trial <- tibble(variable = c("PurPromo6", "PurOtherPromo")) |>
  mutate(rows_in_tidy = map_int(variable, \(v) sum(yj_kept$terms == v)),
       best_lambda  = map_dbl(variable, \(v) yj_best(soap_cluster_raw[[v]])))

yj_trial |>
  gt() |>
  fmt_number(best_lambda, decimals = 4) |>
  cols_label(variable = "Variable", rows_in_tidy = "Powers kept by recipes",
             best_lambda = "Best power in [-5, 5]")
```

Table 17: Trial Yeo-Johnson step on the two promotion shares: the power the recipe keeps, and the best power inside its search range.

Variable	Powers kept by recipes	Best power in [–5, 5]
PurPromo6	0	–4.9999
PurOtherPromo	0	–4.9999

Figure 3 shows the shapes and Table 16 the treatment chosen for each variable. Two different treatments are used, because two different problems cause the skew. Where a variable is strongly skewed but has few zeros (volume and frequency of purchase), the **Yeo-Johnson** transformation (`step_YeoJohnson()`) is used [13]. It learns the power that makes the data most symmetric and, unlike `log()`, it accepts zeros. Where a variable is strongly skewed *and* a large share of households sit at an exact zero (the promotion shares), the **square root** (`step_sqrt()`) is used instead. A first attempt with Yeo-Johnson on those shares left them unchanged, and Table 17 shows why. `recipes` searches for the power between −5 and 5, and for both shares the best power sits on the lower limit (-4.9999), so `recipes` returns no estimate and leaves the column as it is. In other words the search stops at the edge of its range rather than at a real best value. The square root is defined at zero, shrinks the long right tail and keeps the order of the households. Variables below the threshold are left alone, because a transformation that is not needed only makes the result harder to explain.

7.3 Step 3: Standardisation

Euclidean distance adds up squared differences, so a variable measured in thousands (volume) would completely drown out one measured between 0 and 1 (a share). Standardising gives every variable a mean of 0 and a standard deviation of 1, so each has an equal say. Table 18 shows how far apart the scales start.

```
soap_cluster_raw |>
  summarise(across(all_of(cluster_vars), list(mean = mean, sd = sd))) |>
  pivot_longer(everything(), names_to = c("variable", ".value"),
               names_pattern = "(.+)_(mean|sd)") |>
  gt() |>
  fmt_number(c(mean, sd), decimals = 2) |>
  cols_label(variable = "Variable", mean = "Mean", sd = "Standard deviation")
```

Table 18: Scales before standardising.

Variable	Mean	Standard deviation
TotalVol	11,914.77	7,770.37
NoTrans	31.15	17.43
PurPromo6	0.05	0.09
PurOtherPromo	0.03	0.07
NoBrands	3.64	1.58
MaxBrShare	0.37	0.29

Both the transformation and the standardisation are done in one `recipes` recipe, so the same steps could be applied to new households later. The order is deliberate: skewness is corrected first, then the corrected values are standardised.

```
prep_rec <- recipe(~ ., data = soap_cluster_raw) |>
  # keep the ID, but never transform it
  update_role(MemberID, new_role = "id") |>
  step_sqrt(all_of(sqrt_vars)) |>                # zero-heavy skewed shares
  # other strongly skewed variables
  step_YeoJohnson(all_of(yj_vars)) |>
  step_normalize(all_numeric_predictors()) |>   # mean 0, standard deviation 1
  # learn the lambdas, means and sds
  prep()

# the prepared data for Q2c and Q3
soap_prep <- bake(prep_rec, new_data = NULL)
```

```
soap_prep |>
  summarise(across(all_of(cluster_vars), list(mean = mean, sd = sd))) |>
  pivot_longer(everything(), names_to = c("variable", ".value"),
               names_pattern = "(.+)_(mean|sd)") |>
  gt() |>
  fmt_number(c(mean, sd), decimals = 2) |>
  cols_label(variable = "Variable", mean = "Mean", sd = "Standard deviation")
```

Table 19: Scales after the recipe: every variable has mean 0 and standard deviation 1.

Variable	Mean	Standard deviation
TotalVol	0.00	1.00
NoTrans	0.00	1.00
PurPromo6	0.00	1.00
PurOtherPromo	0.00	1.00
NoBrands	0.00	1.00
MaxBrShare	0.00	1.00

```
# Skewness before and after. Standardising does not change skewness,
# so any change here is due to the square root and Yeo-Johnson steps.
skew_after <- soap_prep |>
  summarise(across(all_of(cluster_vars), skewness)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "skew_after")

skew_before |>
  left_join(skew_after, by = "variable") |>
  mutate(method = case_when(variable %in% sqrt_vars ~ "Square root",
```

```
                          variable %in% yj_vars   ~ "Yeo-Johnson",
                          TRUE                    ~ "None")) |>
  gt() |>
  fmt_number(c(skew_before, skew_after), decimals = 2) |>
  cols_label(variable = "Variable", skew_before = "Before",
             skew_after = "After",
             method = "Treatment")
```

Table 20: Skewness before and after the recipe.

Variable	Before	After	Treatment
TotalVol	1.45	0.04	Yeo-Johnson
NoTrans	1.26	0.07	Yeo-Johnson
PurPromo6	2.81	1.04	Square root
PurOtherPromo	5.97	1.43	Square root
NoBrands	0.52	0.52	None
MaxBrShare	0.66	0.66	None

```
soap_prep |>
  pivot_longer(all_of(cluster_vars), names_to = "variable",
               values_to = "value") |>
  ggplot(aes(value)) +
  geom_histogram(bins = 30, fill = pal[2], colour = "white") +
  facet_wrap(~variable, scales = "free") +
  labs(x = "Standardised value", y = "Households")
```

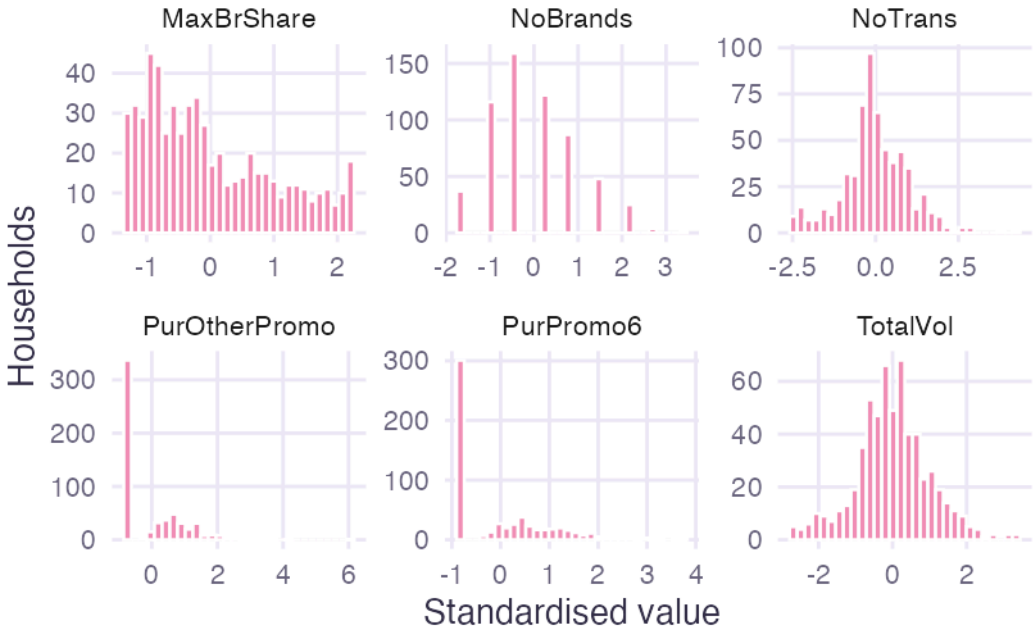


Figure 4: The six variables after transformation and standardising.

7.4 Step 4: Outliers

Even after the transformation, a household can sit far from the rest, and cluster methods are sensitive to such points. A standardised value beyond 3 (more than 3 standard deviations from the mean) is used as the flag. Table 21 counts the flagged households and Figure 5 shows where they sit.

```
outlier_tbl <- soap_prep |>
  summarise(across(all_of(cluster_vars), \(x) sum(abs(x) > 3))) |>
  pivot_longer(everything(), names_to = "variable",
               values_to = "households_beyond_3sd")

n_outlier_hh <- soap_prep |>
  filter(if_any(all_of(cluster_vars), \(x) abs(x) > 3)) |>
  nrow()

outlier_tbl |>
  gt() |>
  cols_label(variable = "Variable",
             households_beyond_3sd = "Households with |z| > 3")
```

Table 21: Households beyond 3 standard deviations on each prepared variable.

Variable	Households with |z| > 3
TotalVol	4
NoTrans	2
PurPromo6	4
PurOtherPromo	7
NoBrands	2
MaxBrShare	0

```
soap_prep |>
  pivot_longer(all_of(cluster_vars), names_to = "variable",
               values_to = "value") |>
  ggplot(aes(variable, value)) +
  geom_boxplot(fill = pal[3], outlier.colour = pal[1]) +
  coord_flip() +
  labs(x = NULL, y = "Standardised value")
```

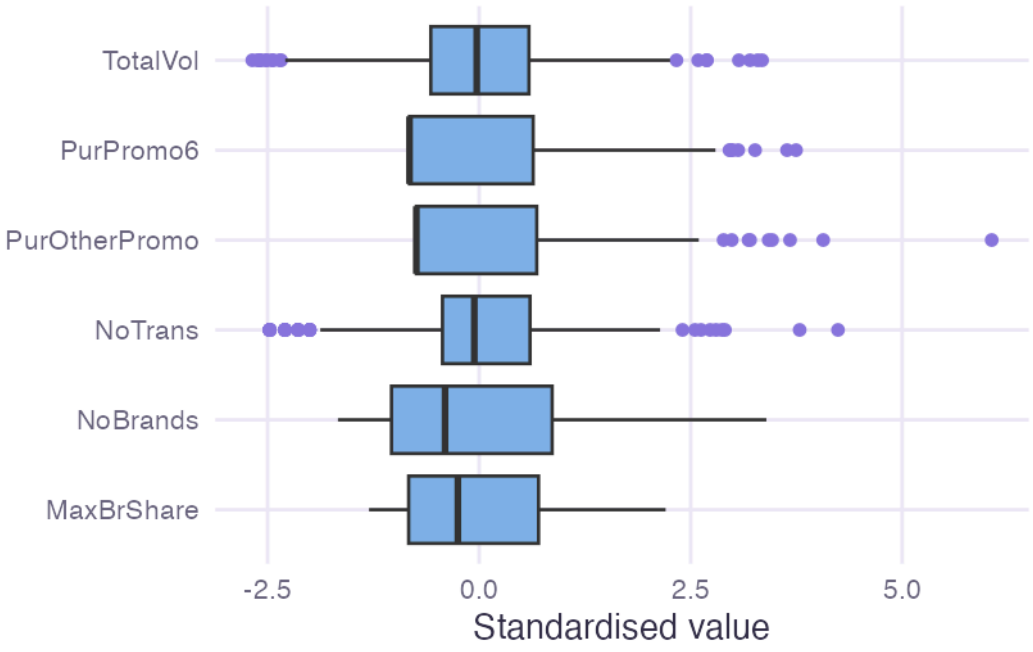


Figure 5: Boxplots of the prepared variables.

```
sk_tab   <- skew_before |> left_join(skew_after, by = "variable")
n_flag   <- length(skewed_vars)
worst_v  <- sk_tab |> slice_max(abs(skew_after), n = 1)
fixed_v  <- sk_tab |> filter(variable %in% yj_vars)
# households extreme ONLY on the square-rooted promotion shares
```

```
n_promo_only <- soap_prep |>
  filter(if_any(all_of(cluster_vars), \(x) abs(x) > 3),
         if_all(all_of(setdiff(cluster_vars, sqrt_vars)), \(x) abs(x) <= 3))
|>
  nrow()
```

Interpretation. Table 20 and Figure 4 show the result. Of the six variables, 4 were strongly skewed before any treatment (|skewness| above 1). For the variables treated with Yeo-Johnson (TotalVol, NoTrans) the skewness fell to between 0.04 and 0.07, which is effectively symmetric. The square root helped the promotion shares (PurPromo6, PurOtherPromo) but did not remove their skew; the most skewed variable after treatment is PurOtherPromo at 1.43. That remaining skew is structural rather than a flaw in the method: around half the households never buy on those promotions, so a large block sits at zero however the scale is stretched. It is therefore reported as a limitation. It also tells the clustering something real, namely that there is a group of households that never respond to promotions.

After the recipe every variable has a mean of 0 and a standard deviation of 1 (Table 19), so no variable dominates a distance only because of its units. Beyond three standard deviations there are 17 households (2.8% of 600) with at least one extreme value, and 11 of them are extreme only on the promotion shares. These households are **kept**. They are genuine heavy promotion users, which is exactly the behaviour a segmentation should be able to find, and removing them would hide the group a promotion-led client most wants to identify. Whether the segments depend on them is tested in Section 8.3.b, where they are removed and the households reclustered.

8 Question 2(c) and 2(d): Clustering the Households

8.1 Choosing the method

The clustering method used is **agglomerative hierarchical clustering**. It works from the bottom up: each household starts as its own cluster, and at each step the two most similar clusters are combined into a bigger cluster until all households belong to one cluster. Unlike k-means, hierarchical clustering does not need the number of clusters to be fixed in advance, and its results can be shown as a dendrogram, which helps in deciding the final number of clusters.

The dissimilarity between households is measured with the **Euclidean distance**, the most commonly used distance measure. Euclidean distance needs numeric variables on a similar scale and is sensitive to outliers. These requirements were met in Section 7, where the six variables were skew-corrected and standardised so that no variable dominates the distance because of its larger range of values.

The linkage method is **Ward's minimum variance method** [14]. At each step it merges the pair of clusters whose merger gives the **smallest increase in the total within-cluster sum of squares**, so it tends to produce compact clusters of similar size. Single linkage was not used because it tends to chain households into long, loose clusters. Complete and average linkage were both run as a check, and Section 8.3.b shows why Ward's method was kept: on these data the other two leave very small or very uneven groups.

8.2 Choosing the number of groups

```
# Cluster on the six variables only (MemberID is a label, not behaviour)
cluster_data <- soap_prep |> select(all_of(cluster_vars))

# Elbow and silhouette using hierarchical clustering with Ward's linkage
p_elbow <- fviz_nbclust(cluster_data, FUNcluster = hcut, method = "wss",
                        k.max = 10, hc_method = "ward.D2") +
  labs(title = "(A) Elbow method")

p_silhouette <- fviz_nbclust(cluster_data, FUNcluster = hcut,
                             method = "silhouette",
                             k.max = 10, hc_method = "ward.D2") +
  labs(title = "(B) Silhouette method")

p_elbow + p_silhouette
```

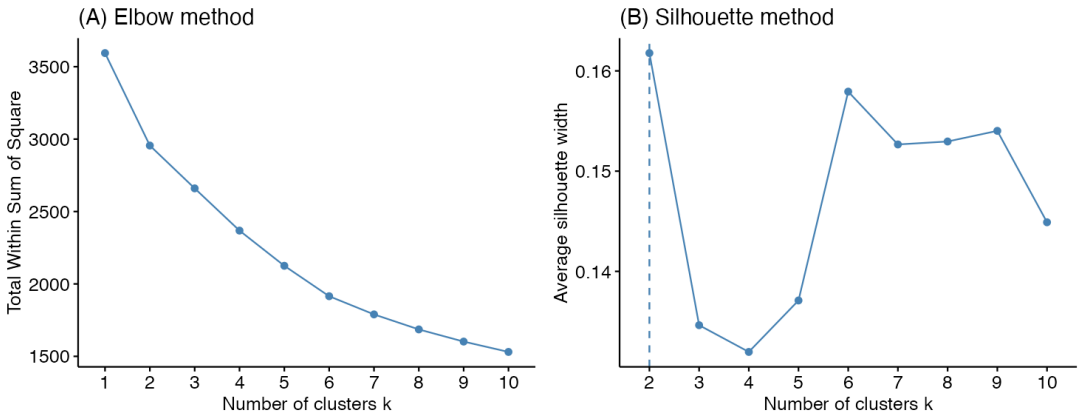


Figure 6: Choosing the number of groups with Ward's linkage: (A) total within-cluster sum of squares and (B) average silhouette width, for 1 to 10 groups.

```
# Hierarchical clustering: Euclidean distance, Ward's linkage
distance_ward <- dist(cluster_data, method = "euclidean")
hc_ward <- hclust(distance_ward, method = "ward.D2")

# Number of clusters selected (justified below)
k_final <- 6

# The dendrogram cut into k_final groups, coloured with the team palette
fviz_dend(hc_ward, k = k_final, show_labels = FALSE, k_colors =
pal[1:k_final],
          main = NULL)
```

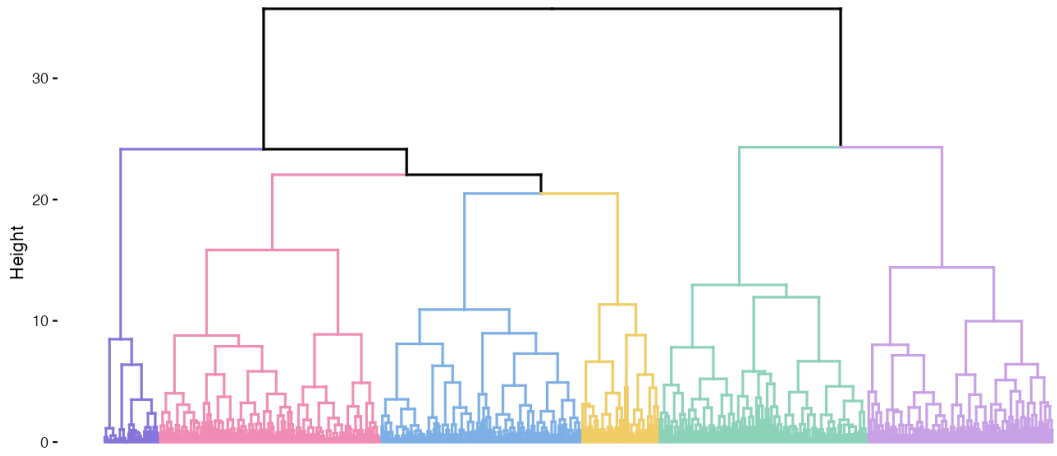


Figure 7: Dendrogram of all households (Ward's linkage), cut into six groups.

```
# For each k, cut the Ward tree and record group sizes and average silhouette
# width
k_compare <- tibble(k = 2:10) |>
  mutate(groups = map(k, \(kk) cutree(hc_ward, k = kk)),
         sizes = map_chr(groups, \(g) paste(table(g), collapse = ", ")),
         smallest = map_int(groups, \(g) min(table(g))),
         average_silhouette = map_dbl(groups, \(g) {
           mean(silhouette(g, distance_ward)[, "sil_width"])
         }))

k_compare |>
  select(-groups) |>
  gt() |>
  fmt_number(average_silhouette, decimals = 3) |>
  cols_label(k = "Groups (k)", sizes = "Households per group",
             smallest = "Smallest group",
             average_silhouette = "Average silhouette")
```

Table 22: Candidate numbers of groups (2 to 10) compared: group sizes and average silhouette width.

Groups (k)	Households per group	Smallest group	Average silhouette
2	351, 249	249	0.162
3	351, 132, 117	117	0.135
4	316, 132, 35, 117	35	0.132
5	140, 132, 35, 176, 117	35	0.137
6	140, 132, 35, 127, 49, 117	35	0.158
7	54, 132, 35, 86, 127, 49, 117	35	0.153
8	54, 132, 35, 86, 127, 49, 65, 52	35	0.153
9	54, 84, 48, 35, 86, 127, 49, 65, 52	35	0.154
10	54, 54, 48, 35, 86, 127, 49, 65, 30, 52	30	0.145

```
sil_at <- \(kk) sprintf("%.3f", k_compare$average_silhouette[k_compare$k ==
kk])
# Best k once the two-group split is set aside, and the range for 7 to 10
best_k_3up <- k_compare |> filter(k >= 3) |> slice_max(average_silhouette)
|>
  pull(k)
sil_7_10   <- range(k_compare$average_silhouette[k_compare$k >= 7])
sizes_of <- \(kk) sort(table(cutree(hc_ward, k = kk)), decreasing = TRUE)
k2 <- sizes_of(2)
# The group of five that is split in two when moving to six
split_parent <- max(table(cutree(hc_ward, k = 5)))
split_kids   <- setdiff(as.integer(table(cutree(hc_ward, k = 6))),
                        as.integer(table(cutree(hc_ward, k = 5))))
smallest6    <- min(table(cutree(hc_ward, k = 6)))
```

Figure 6 shows the elbow and silhouette plots, Figure 7 the dendrogram and Table 22 the group sizes for each candidate. The elbow and silhouette methods do not agree on a clear number of clusters. The silhouette method favours two clusters, where the average silhouette width peaks at 0.162, and it falls sharply at three. However, six clusters are a close second at 0.158, above three, four and five clusters (0.135, 0.132 and 0.137) and above every choice from seven to ten (0.145 to 0.154). Once two groups are set aside, the best number on the silhouette from three to ten is 6. All silhouette values [15] are close to 0, which means many households lie near the border between clusters: the groups overlap rather than being sharply separated. The

elbow plot does not give a clear answer either, as the largest drop is from one to two clusters and the curve flattens out after six.

Six clusters were chosen because the clusters also need to make business sense. The aim of the segmentation is to help XYZ's clients run several promotions, each aimed at a different segment, and to design reward schemes that build brand loyalty. Six segments is a manageable number of separate campaigns for a client to run across a year, and each segment needs a different approach: reward schemes suit loyal households, while households that buy on promotion respond to discounts. With two clusters, the Heavy multi-brand shoppers and the Promo-6 buyers form one cluster of 249 households and everyone else forms a second cluster of 351. That would combine households that buy very differently, such as loyal buyers and light occasional buyers. Moving from five to six clusters splits the largest group of 176 households into 127 loyal households that rarely buy on promotion and 49 loyal households that buy heavily on other promotions. These behave differently and call for different promotions. More clusters would risk very small groups, but with six the smallest still holds 35 households (6% of the panel).

8.3 The final grouping

```
# Cut the Ward tree into k_final groups
ward_groups <- cutree(hc_ward, k = k_final)

# Names chosen from the standardised group means (see the centroid chart below)
segment_names <- c("1" = "Small-brand regular-price buyers",
                   "2" = "Heavy multi-brand shoppers",
                   "3" = "Light occasional buyers",
                   "4" = "Loyal regular-price buyers",
                   "5" = "Loyal promotion buyers",
                   "6" = "Promo-6 buyers")

# Each household's cluster and segment name, used for profiling (2e) and PCA
# (Q3)
soap_clusters <- tibble(MemberID = soap_prep$MemberID,
                        cluster  = factor(ward_groups)) |>
  mutate(segment = unname(segment_names[as.character(cluster)]))
```

Cutting the Ward dendrogram into six clusters gives segments of 35 to 140 households. Each segment was given a label based on its average values on the six variables, and each household's cluster and segment name are stored in `soap_clusters` for the rest of the analysis.

8.3.a Does the result depend on the method?

To check whether the segments depend on the clustering method, k-means was also run with six clusters. K-means assigns each household to the cluster with the nearest centroid, recalculates each centroid as the mean of its cluster, and repeats until the assignments stop changing. Because the starting centroids are random, k-means was run with 25 random starts, keeping the solution with the lowest total within-cluster variation, and the seed is set right before it so the result repeats exactly.

```r
set.seed(55789)   # k-means uses random starting centroids
km_check <- kmeans(cluster_data, centers = k_final, nstart = 25)

ward_vs_km <- table(Ward = ward_groups, Kmeans = km_check$cluster)

# k-means numbers its groups differently, so agreement is read from the largest
# count in each row: the share of households in their best-matching k-means
# group
agreement     <- sum(apply(ward_vs_km, 1, max)) / length(ward_groups)
row_agreement <- apply(ward_vs_km, 1, max) / rowSums(ward_vs_km)

as.data.frame.matrix(ward_vs_km) |>
  as_tibble(rownames = "cluster") |>
  mutate(segment = segment_names[cluster], agreement = row_agreement) |>
  select(segment, everything(), -cluster) |>
  gt() |>
  fmt_percent(agreement, decimals = 0) |>
  tab_spanner(label = "k-means group", columns = as.character(1:k_final)) |>
  cols_label(segment = "Ward segment", agreement = "In best-matching group")
```

Table 23: Cross table of the Ward segments (rows) against the k-means groups (columns).

Ward segment	k-means group 1	2	3	4	5	6	In best-matching group
Small-brand regular-price buyers	4	68	35	8	18	7	49%
Heavy multi-brand shoppers	6	38	1	2	85	0	64%
Light occasional buyers	0	0	0	0	0	35	100%
Loyal regular-price buyers	2	30	0	88	2	5	69%
Loyal promotion buyers	0	0	35	11	0	3	71%
Promo-6 buyers	88	6	0	0	16	7	75%

```
# Repeat k-means 10 times with different random starts: does the agreement
hold?
stability_runs <- map_dbl(1:10, \(i) {
  km  <- kmeans(cluster_data, centers = k_final, nstart = 25)
  tab <- table(ward_groups, km$cluster)
  sum(apply(tab, 1, max)) / length(ward_groups)
})
```

```
# Share of a segment's households in its best-matching k-means group, as text
row_agree_text <- \(seg) {
  percent(row_agreement[names(segment_names)[segment_names == seg]],
          accuracy = 1)
}
```

Table 23 cross-tabulates the two solutions. Overall, 66.5% of households sit in their best-matching k-means group. Agreement is highest for the Light occasional buyers (100%) and the Promo-6 buyers (75%), followed by the Loyal promotion buyers (71%), the Loyal regular-price buyers (69%) and the Heavy multi-brand shoppers (64%). It is lowest for the Small-brand regular-price buyers (49%), whose households are split across several k-means groups. This moderate agreement is consistent with the low silhouette values in Section 8.2: many households lie close to the border between clusters, and the Small-brand regular-price segment is the least sharply defined. Repeating k-means ten times with different random starts gave almost the same agreement each time (65.3% to 66.5%), so the comparison does not depend on one particular set of starting centroids.

8.3.b How stable are the six segments?

The best-match share above flatters the agreement, because each segment is matched to whichever k-means group suits it best. Further checks test the six segments more strictly. The **adjusted Rand index** (ARI) [16] compares two groupings pair by pair: it is the share of household pairs that both groupings put together or both put apart, corrected so that 0 is the agreement expected by chance and 1 means identical groupings. It is computed with `mclust` [11].

```
# (a) best k from 3 to 10 on the silhouette (k = 2 is ruled out above)
best_sil <- k_compare$average_silhouette[k_compare$k == best_k_3up]

# (b) the same distances cut into six groups with two other linkages
linkage_sizes <- map(c(Ward = "ward.D2", Complete = "complete",
                       Average = "average"),
                     \(m) sort(table(cutree(hclust(distance_ward, method
= m),
                                            k = k_final)), decreasing
= TRUE))

# (c) ARI between the Ward segments and k-means
ari_kmeans <- mclust::adjustedRandIndex(ward_groups, km_check$cluster)

# (d) drop households with any |z| > 3, recluster, compare on those left
keep_hh     <- rowSums(abs(cluster_data) > 3) == 0
ward_trim      <-  cutree(hclust(dist(cluster_data[keep_hh,  ]),  method  =
"ward.D2"),
                      k = k_final)
ari_trimmed <- mclust::adjustedRandIndex(ward_groups[keep_hh], ward_trim)
```

```r
# (e) average silhouette width within each segment
sil_ward   <- silhouette(ward_groups, distance_ward)
sil_seg_of <- unname(segment_names[as.character(sil_ward[, "cluster"])])
sil_by_seg <- tibble(segment = sil_seg_of, width = sil_ward[, "sil_width"])
|>
  summarise(width = mean(width), .by = segment) |>
  arrange(desc(width))

robust_tbl <- bind_rows(
  tibble(group  = "Number of groups",
         check  = "Best k from 3 to 10 (average silhouette)",
         result = paste0("k = ", best_k_3up, " (", sprintf("%.3f", best_sil),
                         ")")),
  tibble(group  = "Linkage method (sizes at k = 6)",
         check  = names(linkage_sizes),
         result = map_chr(linkage_sizes, \(x) paste(x, collapse = ", "))),
  tibble(group  = "Agreement (adjusted Rand index)",
         check  = c("Ward against k-means, all households",
                    paste0("Ward against Ward without the ", sum(!keep_hh),
                           " extreme households (n = ", sum(keep_hh), ")")),
         result = sprintf("%.3f", c(ari_kmeans, ari_trimmed))),
  tibble(group  = "Average silhouette width by segment",
         check  = sil_by_seg$segment,
         result = sprintf("%.3f", sil_by_seg$width))
)

robust_tbl |>
  gt(groupname_col = "group") |>
  cols_label(check = "Check", result = "Result")
```

Table 24: Stability checks on the six Ward segments.

Check	Result
Number of groups	
Best k from 3 to 10 (average silhouette)	k = 6 (0.158)
Linkage method (sizes at k = 6)	
Ward	140, 132, 127, 117, 49, 35
Complete	254, 114, 104, 96, 31, 1
Average	541, 34, 20, 3, 1, 1
Agreement (adjusted Rand index)	
Ward against k-means, all households	0.389
Ward against Ward without the 17 extreme households (n = 583)	0.619
Average silhouette width by segment	
Light occasional buyers	0.467
Loyal regular-price buyers	0.229
Loyal promotion buyers	0.140
Promo-6 buyers	0.130
Heavy multi-brand shoppers	0.129
Small-brand regular-price buyers	0.073

```
# Look-ups for the paragraph below
complete_min <- min(linkage_sizes$Complete)
average_max  <- max(linkage_sizes$Average)
sil_weakest  <- slice_min(sil_by_seg, width)
sil_strongest <- slice_max(sil_by_seg, width)
```

Table 24 gives the results.

- **Six is the best number of groups once two is set aside.** From three to ten groups the average silhouette is highest at 6.
- **Ward's linkage is the only one of the three that gives usable segments.** On the same distances, complete linkage leaves a group of only 1 household and average linkage puts 541 of the 600 households in one group. Neither could be turned into six campaigns.
- **Agreement with k-means is moderate.** The ARI is 0.389, so k-means finds a similar structure but moves many households between neighbouring groups. This is a stricter and less flattering figure than the 66.5% best-match share.
- **Most of the structure holds without the extreme households.** Removing the 17 households with a standardised value beyond 3 (Section 7) and clustering the other 583 again

gives an ARI of 0.619 with the original segments. Most of the structure survives, although some households change group.

- **Some segments are much sharper than others.** The average silhouette width runs from 0.467 for the Light occasional buyers to 0.073 for the Small-brand regular-price buyers. The first is a clearly separate group, while the second sits close to its neighbours, which matches its low k-means agreement above.

So the six segments have real cores, but their borders are not sharp, and the weakest segments should be read as tendencies rather than fixed types.

8.4 Describing each group (2d)

```
# Attach each household's segment to its original (untransformed) values
original_segment <- soap_clusters |>
  left_join(soap_clean |> select(MemberID, all_of(cluster_vars)),
            by = "MemberID")

# Group sizes and means, plus an all-households row for comparison
segment_means <- original_segment |>
  group_by(cluster, segment) |>
  summarise(households = n(), across(all_of(cluster_vars), mean),
            .groups = "drop") |>
  bind_rows(original_segment |>
              summarise(households = n(), across(all_of(cluster_vars), mean))
|>
               mutate(segment = "All households"))

segment_means |>
  select(-cluster) |>
  gt() |>
  fmt_number(c(TotalVol, NoTrans, NoBrands), decimals = 1) |>
  fmt_percent(c(PurPromo6, PurOtherPromo, MaxBrShare), decimals = 0) |>
  cols_label(segment = "Segment", households = "Households",
             TotalVol = "Total volume", NoTrans = "Transactions",
             PurPromo6 = "Share on promo 6",
             PurOtherPromo = "Share on other promos",
             NoBrands = "Brands bought", MaxBrShare = "Share to top brand")
|>
  tab_style(style = cell_text(weight = "bold"),
            locations = cells_body(rows = segment == "All households"))
```

Table 25: How each segment buys bath soap, in original units, with the all-household average in the last row.

Segment	Households	Total volume	Transactions	Share on promo 6	Share on other promos	Brands bought	Share to top brand
Small-brand regular-price buyers	140	10,425.7	28.1	1%	4%	3.2	21%
Heavy multi-brand shoppers	132	17,076.9	50.6	4%	2%	5.0	24%
Light occasional buyers	35	1,321.4	3.5	0%	0%	1.8	46%
Loyal regular-price buyers	127	12,890.7	24.8	2%	0%	3.2	66%
Loyal promotion buyers	49	13,049.9	23.7	2%	15%	2.7	77%
Promo-6 buyers	117	9,506.9	31.2	19%	4%	4.1	20%
All households	**600**	**11,914.8**	**31.2**	**5%**	**3%**	**3.6**	**37%**

```
soap_prep |>
  left_join(soap_clusters, by = "MemberID") |>
  group_by(segment) |>
  summarise(across(all_of(cluster_vars), mean)) |>
  pivot_longer(-segment, names_to = "variable", values_to = "mean_z") |>
  mutate(segment  = factor(segment, levels = segment_names),
         variable = factor(variable, levels = rev(cluster_vars))) |>
  ggplot(aes(mean_z, variable, fill = mean_z > 0)) +
  geom_col(show.legend = FALSE) +
  geom_vline(xintercept = 0, colour = "grey40") +
  facet_wrap(~segment, ncol = 2) +
  scale_fill_manual(values = c(`TRUE` = pal[1], `FALSE` = pal[4])) +
  labs(title = "Segment centroids on the six purchase variables",
       x = "Standardised mean (0 = average household)", y = NULL)
```

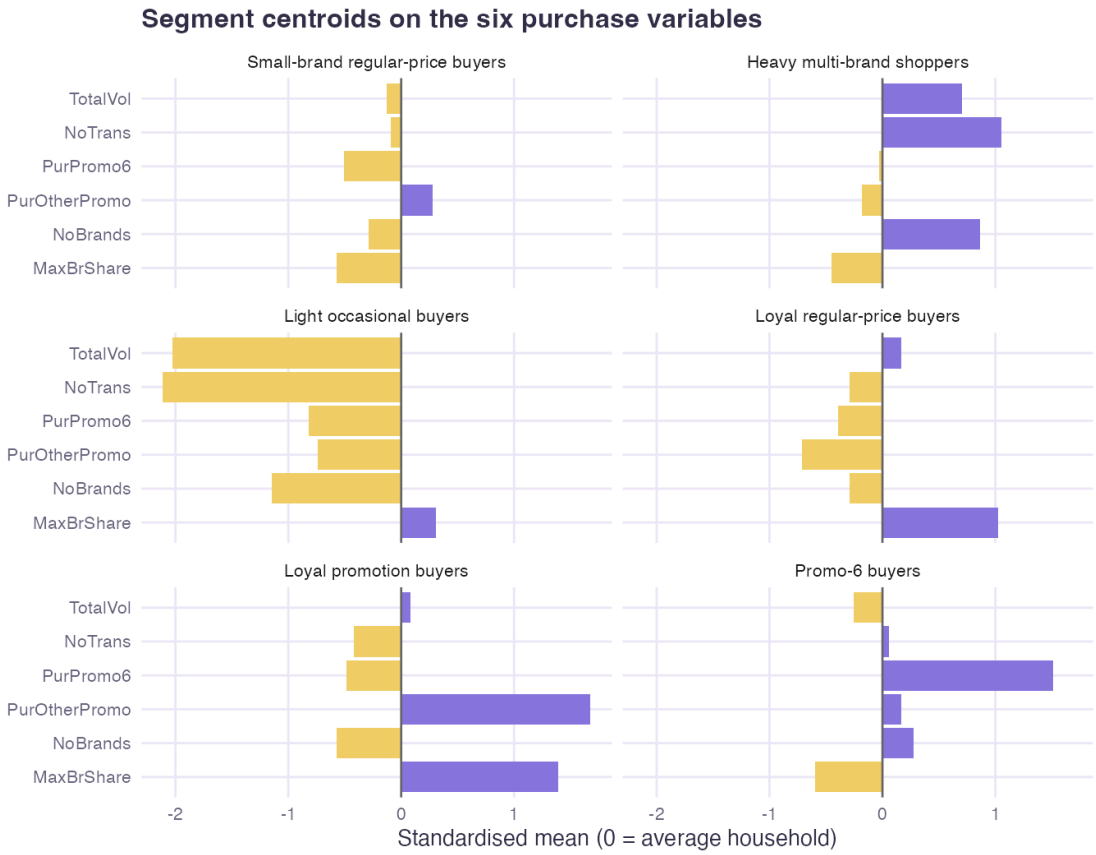


Figure 8: Segment centroids: the mean of each standardised variable within each segment (0 = the average household).

```
# Look-ups for the paragraphs below, so every number comes from the table above
seg_mean <- \(seg, col) segment_means[[col]][segment_means$segment == seg]
all_mean <- \(col) seg_mean("All households", col)
seg_size <- \(seg) seg_mean(seg, "households")
fmt_pct  <- \(x) percent(x, accuracy = 1)
```

```
fmt_vol  <- \(x) comma(x, accuracy = 100)
n_hh     <- nrow(soap_prep)

# BrOthers was not clustered on, so it is only used to profile the segments
brothers_seg <- soap_clusters |>
  left_join(select(soap, MemberID, BrOthers, MaxBrShare), by = "MemberID")
|>
  summarise(brothers = mean(BrOthers), zero_max = sum(near(MaxBrShare, 0)),
            .by = segment)
brothers_of <- \(seg) brothers_seg$brothers[brothers_seg$segment == seg]
zero_of     <- \(seg) brothers_seg$zero_max[brothers_seg$segment == seg]
zero_segs   <- brothers_seg |> filter(zero_max > 0) |> arrange(desc(zero_max))
```

Table 25 gives each segment's size and average purchase behaviour in original units, with the average for all households in the last row. Figure 8 shows the same segments on the standardised scale: bars to the right are above the average household and bars to the left are below it, so the longest bars show what makes each segment distinctive.

Small-brand regular-price buyers (140 households, 23% of the panel). These households buy close to the average amount of soap, about 10,400 units over 28 transactions, and rarely buy on promotion (1% of volume on promotion 6 and 4% on other promotions). Only 21% of their volume goes to their top named brand, well below the average of 37%, so the eight named brands win little of their spending. Instead, 71% of their volume goes to `BrOthers`, the brands outside the eight named, the highest of any segment. `BrOthers` was not a clustering variable, so this is a profiling observation. It is not evidence of loyalty either, because `BrOthers` pools many small brands and cannot show whether a household stays with one of them. Because they rarely buy on promotion, discounts are unlikely to change their habits.

Heavy multi-brand shoppers (132 households, 22%). This segment buys the most soap, about 17,100 units, which is 43% more than the average household, and buys most often, with about 51 transactions against an average of 31. They spread their volume over the most brands (5 on average against 3.6 overall) with only 24% going to their top brand. Their volume is large but divided between brands, so reward schemes that encourage them to keep buying the same brand suit this segment.

Light occasional buyers (35 households, 6%). These households buy very little, about 1,300 units over only 3.5 transactions, only 11% of the average volume. They did not buy on promotion at all and bought the fewest brands (1.8 against 3.6). Given their small volume they should be a low priority for promotion spending.

Loyal regular-price buyers (127 households, 21%). These households buy slightly more than average, about 12,900 units, in fewer transactions (25 against 31). They give 66% of their volume to one brand and almost never buy on promotion (2% on promotion 6 and 0% on others). Since they already buy their favourite brand without discounts, reward schemes that maintain their loyalty suit them better than price promotions, which would only give away margin.

Loyal promotion buyers (49 households, 8%). This is the most loyal segment, with 77% of volume going to one brand and only 2.7 brands bought on average. They buy an average

amount, about 13,000 units, but 15% of their volume is bought on other promotions, about 4.5 times the average of 3%. Because this segment is loyal and also responds to promotions, promotions on their favourite brand suit them.

Promo-6 buyers (117 households, 20%). These households buy slightly less than average, about 9,500 units, at an average frequency. They are among the least loyal, with only 20% of volume going to their top brand across 4.1 brands, and 19% of their volume is bought on promotion 6, about 3.6 times the average of 5%. Their low `MaxBrShare` is partly a small-brand effect too, since 68% of their volume goes to `BrOthers`. Promotion 6 offers timed at different points of the year suit this segment best.

Where the `MaxBrShare` = 0 households went. Question 1 showed that the 22 households with `MaxBrShare` = 0 buy only one brand, which sits inside `BrOthers`. They are not spread evenly: 9 are in the Small-brand regular-price buyers, 9 are in the Light occasional buyers, 3 are in the Promo-6 buyers, 1 is in the Loyal promotion buyers. On `MaxBrShare` they look like the least loyal households, so the clustering places most of them in low-concentration segments even though each one is fully loyal to a small brand. This is the limit that Question 1 item 7 warned about, and it is one more reason the low `MaxBrShare` of the Small-brand regular-price and Promo-6 buyers should not be read as plain disloyalty.

9 Question 2(e): Profiling the Groups

The segments in Section 8 were built from purchase behaviour only. Demographics were deliberately left out of the clustering, so that the groups describe *how* households buy rather than *who* they are. This section brings the demographics back to answer the client's next question: who are the households in each segment, and how can they be reached?

9.1 Which three demographics, and why

Each variable was chosen because a difference between segments on it leads to a different business action (Table 26).

```
# The three demographics used to profile the segments
profile_vars <- c("SEC", "HS", "AGE")

# Missing values per profile variable (zeros were turned into NA in Section 7)
profile_missing <- soap_clean |>
  summarise(across(all_of(profile_vars), \(x) sum(is.na(x)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "n_missing")

tibble(
  variable = profile_vars,
  type     = c("Ordinal, 1 high to 4 low", "Numeric, number of members",
               "Ordinal, 1 young to 4 old"),
  action   = c("Price point and value or premium range",
               "Pack size and multi-packs",
               "Media channel and message")
) |>
  left_join(profile_missing, by = "variable") |>
```

```
  gt() |>
  cols_label(variable = "Variable", type = "Type",
             action = "Business lever it informs",
             n_missing = "Missing households")
```

Table 26: The three profiling demographics and the business lever each one informs.

Variable	Type	Business lever it informs	Missing households
SEC	Ordinal, 1 high to 4 low	Price point and value or premium range	0
HS	Numeric, number of members	Pack size and multi-packs	68
AGE	Ordinal, 1 young to 4 old	Media channel and message	0

```
other_missing <- soap_clean |>
  summarise(across(c(FEH, MT, SEX, EDU, CHILD, CS, Affluence),
                   \(x) sum(is.na(x))))
share_female <- mean(soap_clean$SEX == 2, na.rm = TRUE)
# Spearman correlation, because SEC is ordinal
sec_aff_cor     <-   cor(soap_clean$SEC,   soap_clean$Affluence,   method   =
"spearman",
                     use = "complete.obs")
```

- **SEC (socioeconomic class)** tells a manufacturer what price a segment can bear. SEC is also one of the strata the panel was sampled on, so all 600 households have a value and each class holds exactly a quarter of the panel, which makes any tilt within a segment easy to read.
- **HS (household size)** is the demographic most directly tied to how much soap a household uses, so it tells the client which segments suit larger packs or multi-packs. 68 households have no size recorded and are left out of the HS summaries only.
- **AGE (age of the homemaker)** tells an advertising agency where and how to speak to each segment. Every household has a value.

Why not the others. `Affluence` overlaps with SEC (Spearman correlation −0.61; SEC runs from 1 high to 4 low, so a negative value means wealthier households sit in the higher classes) and has 68 missing households. `MT` (native language) and `CHILD` are codes with no labels in the data guide, so a difference could not be turned into an action. `SEX` barely varies: 96% of homemakers with a recorded value are female. `EDU`, `FEH` and `CS` are missing for 73, 69 and 99 households and are less directly linked to a bath soap decision. The appendix tests this choice against the data.

Missing values. No household is dropped from the segmentation. For HS, each summary and test uses only the households that have a value, and the number used is shown.

9.2 Profile by group

```
# Attach the three demographics (and TotalVol, to link size to volume) to each
# household's segment
profile_df <- soap_clusters |>
  select(MemberID, cluster, segment) |>
  left_join(soap_clean |> select(MemberID, all_of(profile_vars), TotalVol),
            by = "MemberID") |>
  mutate(
    # Readable labels for the ordinal codes
    SEC = factor(SEC, levels = 1:4, labels = c("1 High", "2", "3", "4 Low")),
    AGE = factor(AGE, levels = 1:4, labels = c("1 Young", "2", "3", "4 Old")),
    segment = factor(segment, levels = segment_names)
  )

# One chart function for both categorical variables, so they look the same
plot_props <- function(data, var, title) {
  data |>
    filter(!is.na({{ var }})) |>
    count(segment, {{ var }}) |>
    group_by(segment) |>
    mutate(prop = n / sum(n)) |>                # share within each segment
    ungroup() |>
    ggplot(aes(prop, fct_rev(segment), fill = {{ var }})) +
    geom_col(colour = "white") +
    scale_x_continuous(labels = percent) +
    scale_fill_manual(values = pal) +
    labs(title = title, y = NULL, x = "Share of households in the segment",
         fill = NULL)
}

# Per-segment summary used in the text and in the summary table
seg_profile <- profile_df |>
  group_by(segment) |>
  summarise(households = n(),
            sec_high  = mean(as.integer(SEC) <= 2),     # SEC 1 or 2
            sec_low   = mean(as.integer(SEC) == 4),     # SEC 4
            hs_n      = sum(!is.na(HS)),
            hs_mean   = mean(HS, na.rm = TRUE),
            hs_median = median(HS, na.rm = TRUE),
            age_old   = mean(AGE == "4 Old"),
            volume    = mean(TotalVol))
```

```
# Cramer's V (Cramer, 1946): chi-squared rescaled to run from 0 to 1
cramers_v <- function(tab, stat) sqrt(stat / (sum(tab) * (min(dim(tab)) - 1)))

# Chi-squared test plus Cramer's V for one categorical variable
```

```
chisq_row <- function(data, var) {
  d   <- data |> filter(!is.na(.data[[var]]))
  tab <- table(d$segment, d[[var]])
  # simulated p-value, see below
  res <- chisq.test(tab, simulate.p.value = TRUE, B = 5000)
  tibble(variable = var, test = "Chi-squared", n = sum(tab),
         p_value = res$p.value,
         effect = cramers_v(tab, unname(res$statistic)),
         effect_measure = "Cramer's V")
}

# only households with a recorded size
hs_df <- profile_df |> filter(!is.na(HS))

set.seed(55789)   # the simulated p-values are random, so fix them
kw_hs <- kruskal.test(HS ~ segment, data = hs_df)

profile_tests <- bind_rows(
  chisq_row(profile_df, "SEC"),
  tibble(variable = "HS", test = "Kruskal-Wallis", n = nrow(hs_df),
         p_value = kw_hs$p.value,
         # epsilon squared
         effect = unname(kw_hs$statistic) / (nrow(hs_df) - 1),
         effect_measure = "Epsilon squared"),
  chisq_row(profile_df, "AGE")
)

# Small helpers for the text
seg_prof <- \(seg, col) seg_profile[[col]][seg_profile$segment == seg]
fmt_p    <- \(p) if_else(p < 0.001, "p < 0.001", paste("p =", sprintf("%.3f",
                                                                      p)))
test_row <- \(var) profile_tests |> filter(variable == var)
```

9.2.a Socioeconomic class

```
plot_props(profile_df, SEC, "Socioeconomic class within each segment")
```

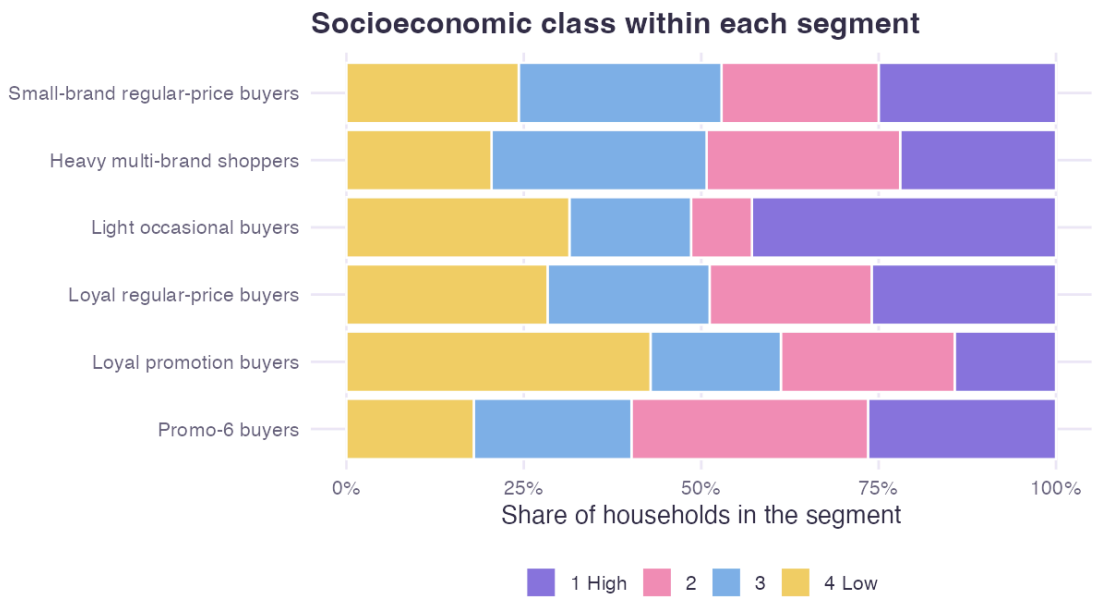


Figure 9: Socioeconomic class within each segment (1 = highest, 4 = lowest). Across the panel each class holds 25% of households.

Figure 9 shows the class mix. Because the panel is stratified on SEC, every class holds 25% of households overall, so a segment that departs from a quarter in a class genuinely leans that way. The **Loyal promotion buyers** have the most lower-class households: 43% are in SEC 4. Together with their high use of other promotions (Section 8), this points to a budget-conscious group that stays with one brand but buys more of it when it is on offer, so price promotions *on their favourite brand* reach them well. The **Promo-6 buyers** lean the other way, with 60% in SEC 1 or 2 against 50% overall: their heavy use of promotion 6 looks like a deal-seeking habit rather than a sign that they cannot afford full price, so the *timing* of promotion 6 matters more than a cheaper product. The test confirms a real but small difference (p = 0.010, Cram<U+00E9>r's V = 0.131), so SEC is a useful secondary guide, not a targeting rule on its own.

9.2.b Household size

```
hs_df |>
  ggplot(aes(HS, fct_rev(segment), fill = segment)) +
  geom_boxplot(show.legend = FALSE) +
  stat_summary(fun = mean, geom = "point", shape = 4, size = 3) +
  scale_fill_manual(values = pal) +
  labs(title = "Household size within each segment",
      subtitle = paste0("Households with a recorded size: n = ", nrow(hs_df)),
       y = NULL, x = "Members in the household")
```

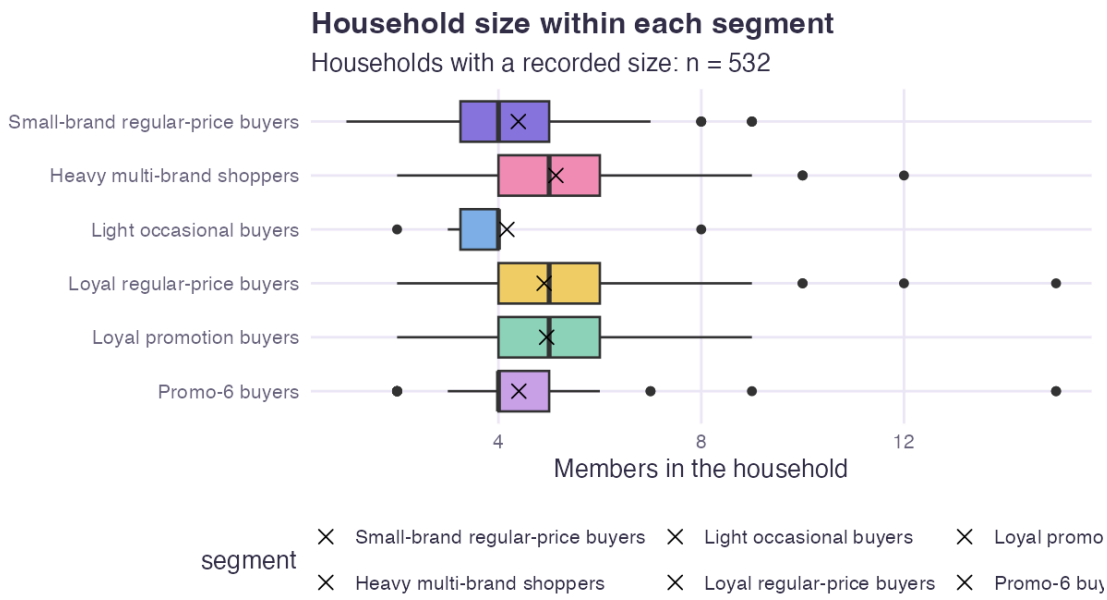


Figure 10: Household size within each segment (households with a recorded size only). The cross marks the mean.

Figure 10 shows household size. The **Heavy multi-brand shoppers** have the largest households (5.1 members on average against 4.7 overall) and also buy the most soap, which fits the simple logic that more people use more soap. For a client this is the natural segment for family packs and multi-packs, ideally tied to one brand to counter their habit of spreading volume across brands. The **Small-brand regular-price buyers** and **Promo-6 buyers** have the smallest households (4.4 and 4.4 members), so standard packs suit them. The difference is statistically significant but small ($p < 0.001$, epsilon squared = 0.039).

One pattern needs a warning: only 6 of the 35 **Light occasional buyers** have a recorded household size. The largest share of the 68 households with no household size (which also lack education, language, gender, TV and Affluence, Section 7) falls into this segment (29 of them), so it may partly reflect households with incomplete panel records rather than a true customer type, so its household size profile should not be relied on. Its `SEC` and `AGE` are recorded for every household, so those two comparisons still hold.

9.2.c Age of the homemaker

```
plot_props(profile_df, AGE, "Age of the homemaker within each segment")
```

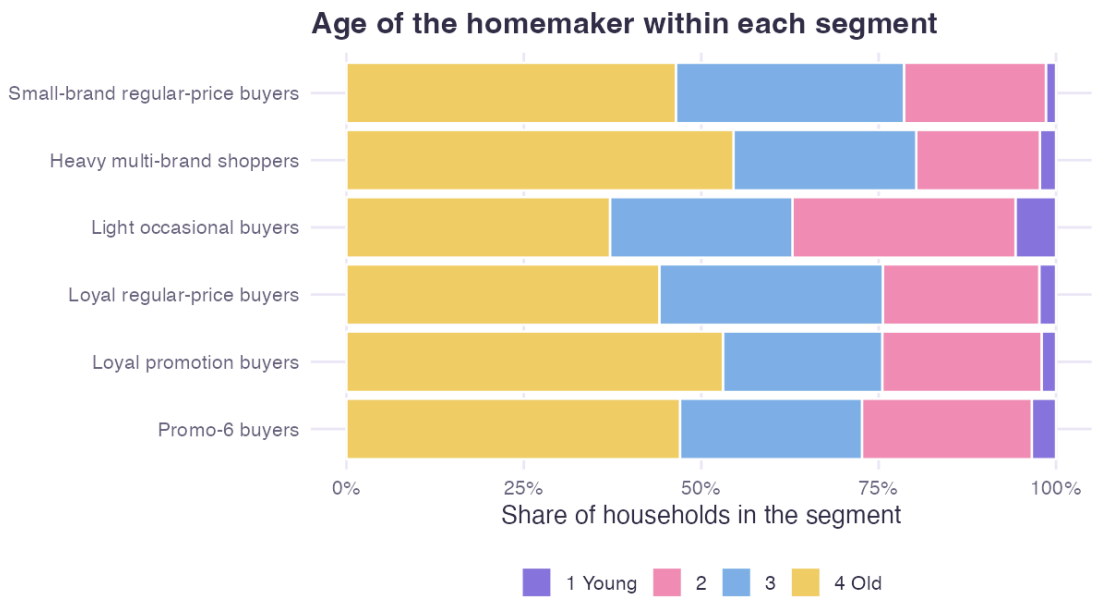


Figure 11: Age of the homemaker within each segment (1 = youngest, 4 = oldest).

Figure 11 shows that age does **not** separate the segments (p = 0.757, Cram<U+00E9>r's V = 0.078). Older homemakers are the largest group in every segment, from 37% to 55% in the oldest class, and the differences in the chart are within what chance would produce. The practical conclusion is that age gives no reason to reach one segment through a different media channel from another.

9.3 Are the differences real?

A chart can show a difference that is only noise, so each variable is tested. SEC and AGE are categorical, so a **chi-squared test of independence** checks whether the mix of classes differs between segments. Some cells are small (few households have the youngest age code), so the p-value is simulated rather than taken from the chi-squared table, which keeps it valid when expected counts are below 5. HS is a count with a long right tail, so the **Kruskal-Wallis test**, which compares segments on ranks rather than means, is used instead of ANOVA. With 600 households even a tiny difference can be "significant", so an effect size is also reported: Cram<U+00E9>r's V [17] for the categorical variables and epsilon squared for HS, both running from 0 (no association) to 1.

```
profile_tests |>
  select(variable, test, n, p_value, effect, effect_measure) |>
  gt() |>
  fmt_number(effect, decimals = 3) |>
  fmt_number(p_value, decimals = 4) |>
  cols_label(variable = "Variable", test = "Test", n = "Households used",
             p_value = "p-value", effect = "Effect size",
             effect_measure = "Measure")
```

Table 27: Do the segments differ on each demographic?

Variable	Test	Households used	p-value	Effect size	Measure
SEC	Chi-squared	600	0.0102	0.131	Cramer's V
HS	Kruskal-Wallis	532	0.0009	0.039	Epsilon squared
AGE	Chi-squared	600	0.7574	0.078	Cramer's V

Table 27 gives the results. SEC and household size differ significantly between segments, but both effects are small, and age does not differ at all. **The segments are therefore only weakly visible in demographics.** That is a finding in itself, and it supports XYZ's move to behavioural segmentation: a client could not have found these segments from demographics alone, so targeting should run on purchase data (panel or loyalty card records), with SEC and household size used to fine-tune the offer.

Where these differences come from (standardised residuals for SEC and AGE, and pairwise tests for household size) and a check of every demographic against the three chosen are in the Further checks part of the appendix.

9.4 Summary profile of each segment

```
seg_profile |>
  select(segment, households, sec_high, sec_low, hs_median, hs_mean, age_old)
|>
  gt() |>
  fmt_percent(c(sec_high, sec_low, age_old), decimals = 0) |>
  fmt_number(hs_mean, decimals = 1) |>
  cols_label(segment = "Segment", households = "Households",
             sec_high = "SEC 1 or 2", sec_low = "SEC 4",
             hs_median = "Median size", hs_mean = "Mean size",
             age_old = "Homemaker AGE 4") |>
  tab_spanner(label = "Socioeconomic class", columns = c(sec_high, sec_low))
|>
  tab_spanner(label = "Household size", columns = c(hs_median, hs_mean))
```

Table 28: Demographic profile of each segment.

Segment	Households	Socioeconomic class		Household size		Homemaker AGE 4
		SEC 1 or 2	SEC 4	Median size	Mean size	
Small-brand regular-price buyers	140	47%	24%	4	4.4	46%
Heavy multi-brand shoppers	132	49%	20%	5	5.1	55%
Light occasional buyers	35	51%	31%	4	4.2	37%
Loyal regular-price buyers	127	49%	28%	5	4.9	44%
Loyal promotion buyers	49	39%	43%	5	5.0	53%
Promo-6 buyers	117	60%	18%	4	4.4	47%

Table 28 brings the profile together. Putting purchase behaviour (Section 8) and demographics together:

- **Heavy multi-brand shoppers:** the largest households and the most volume. Family packs and multi-packs, combined with a reward scheme that concentrates their spending on one brand.
- **Loyal promotion buyers:** the most lower-class households (43% in SEC 4). Value packs and price promotions on the brand they already favour.
- **Promo-6 buyers:** more higher-class households than average (60% in SEC 1 or 2). Promotion 6 offers spread through the year; a cheaper product is not needed.
- **Loyal regular-price buyers:** demographically close to the average household. Reward schemes rather than discounts, which they do not need.

- **Small-brand regular-price buyers:** close to average with smaller households, so demographics give little extra help in finding them; they are best reached through purchase data, with trial and visibility to win them from small brands.
- **Light occasional buyers:** too small, too light and too poorly recorded (29 of 35 have no household size, education or TV record) to target separately.

10 Question 3: Dimension Reduction with PCA

Question 2 described each household with six purchase behaviour variables. **Principal component analysis (PCA)** asks whether those six can be summarised by fewer new variables, called components, without losing much information. Each component is a weighted combination of the six: the first captures as much of the variation between households as possible, the second captures as much of what is left as possible while being uncorrelated with the first, and so on. If the six variables overlap, a few components carry most of the information, which makes the households easier to describe and to plot.

10.1 Is PCA worth doing on these variables?

PCA can only compress variables that move together. If the six were unrelated, every component would carry about the same share and nothing would be gained.

```
pca_cor <- soap_prep |>
  select(all_of(cluster_vars)) |>
  cor()

# The strongest pair, and each variable's largest correlation with any other
top_pair <- as_tibble(pca_cor, rownames = "var1") |>
  pivot_longer(-var1, names_to = "var2", values_to = "r") |>
  filter(var1 < var2) |>                      # each pair once, no diagonal
  slice_max(abs(r), n = 1)

loner <- as_tibble(pca_cor, rownames = "variable") |>
  pivot_longer(-variable, names_to = "other", values_to = "r") |>
  filter(variable != other) |>
  group_by(variable) |>
  summarise(max_abs_r = max(abs(r))) |>
  slice_min(max_abs_r, n = 1)

as_tibble(pca_cor, rownames = "variable") |>
  gt() |>
  fmt_number(-variable, decimals = 2) |>
  data_color(columns = -variable,
             palette = c(pal_lo, "#F9E1EB", pal_hi),
             domain = c(-1, 1), autocolor_text = FALSE)
```

Table 29: Correlations between the six prepared variables (the same data the clustering used).

variable	TotalVol	NoTrans	PurPromo6	PurOther-Promo	No-Brands	MaxBr-Share
TotalVol	1.00	0.64	0.02	0.05	0.28	0.02
NoTrans	0.64	1.00	0.20	0.13	0.56	–0.24
PurPromo6	0.02	0.20	1.00	0.13	0.22	–0.31
PurOther-Promo	0.05	0.13	0.13	1.00	0.03	0.00
No-Brands	0.28	0.56	0.22	0.03	1.00	–0.23
MaxBr-Share	0.02	–0.24	–0.31	0.00	–0.23	1.00

Table 29 shows the correlations. The strongest relationship is between NoTrans and TotalVol (r = 0.64), so there is real overlap for PCA to use. At the other extreme, PurOtherPromo is almost independent of the rest (its largest correlation with any other variable is 0.13), so it cannot be folded into the others and is likely to need a component of its own.

10.2 Running the PCA

The PCA follows the tidymodels recipe pattern [3], [4]: a recipe with `step_normalize()` and `step_pca()`, estimated with `prep()`, and checked against base R's `prcomp()`.

```
pca_data <- soap_prep |> select(MemberID, all_of(cluster_vars))

pca_rec <- recipe(~ ., data = pca_data) |>
  # keep the ID, but leave it out of the PCA
  update_role(MemberID, new_role = "id") |>
  step_normalize(all_numeric_predictors()) |>     # see the note below
  # keep all six for now
  step_pca(all_numeric_predictors(), num_comp = length(cluster_vars),
           id = "pca") |>
  prep()

# What step_normalize learned: for already standardised data, means are 0 and
# sds are 1
norm_check <- tidy(pca_rec, number = 1) |>
  group_by(statistic) |>
  summarise(furthest = max(abs(value - if_else(statistic == "mean", 0, 1))))

# Cross-check against base R: prcomp() should give the same variances
pca_base <- prcomp(select(pca_data, all_of(cluster_vars)))
```

Is `step_normalize()` needed again? PCA must be run on standardised data, otherwise the variable with the largest units (volume) would dominate the first component only because its

numbers are bigger. `soap_prep` was already standardised in Section 7, so this step changes nothing here: the means it learned differ from 0, and the standard deviations from 1, by at most 6e-15, which is rounding error. It is kept so that the recipe is safe on its own if it is ever run on unprepared data. All six components are kept at this stage so that the number to retain is chosen from the evidence below rather than fixed in advance.

10.3 How many components to keep

```
pca_var <- tidy(pca_rec, id = "pca", type = "variance") |>
  pivot_wider(names_from = terms, values_from = value) |>
  transmute(component,
            eigenvalue = variance,
            percent    = `percent variance`,
            cumulative = `cumulative percent variance`)

# The variances match base R's prcomp() (eigenvalue = sd squared)
same_as_prcomp <- isTRUE(all.equal(pca_var$eigenvalue, pca_base$sdev^2,
                                   check.attributes = FALSE))

pca_var |>
  gt() |>
  fmt_number(eigenvalue, decimals = 3) |>
  fmt_number(c(percent, cumulative), decimals = 1, pattern = "{x}%") |>
  cols_label(component = "Component", eigenvalue = "Eigenvalue",
             percent = "Variance explained", cumulative = "Cumulative")
```

Table 30: Variance explained by each component.

Component	Eigenvalue	Variance explained	Cumulative
1	2.186	36.4%	36.4%
2	1.245	20.7%	57.2%
3	1.008	16.8%	74.0%
4	0.673	11.2%	85.2%
5	0.638	10.6%	95.8%
6	0.251	4.2%	100.0%

```
# Kaiser: eigenvalue above 1
n_kaiser <- sum(pca_var$eigenvalue > 1)
# enough components for 70% of the variation
n_70     <- which(pca_var$cumulative >= 70)[1]
# the decision (explained below)
n_keep   <- n_70

scree <- pca_var |>
  ggplot(aes(component, eigenvalue)) +
```

```
  geom_hline(yintercept = 1, linetype = "dashed", colour = "grey55") +
  geom_line(colour = pal[1]) +
   geom_point(aes(colour = component <= n_keep), size = 3, show.legend =
FALSE) +
  scale_colour_manual(values = c(`TRUE` = pal[1], `FALSE` = pal_grey)) +
  scale_x_continuous(breaks = pca_var$component) +
  labs(title = "Scree plot", x = "Component", y = "Eigenvalue")

cumul <- pca_var |>
  ggplot(aes(component, cumulative / 100)) +
  geom_col(aes(fill = component <= n_keep), show.legend = FALSE) +
  geom_hline(yintercept = 0.7, linetype = "dashed", colour = "grey55") +
  scale_fill_manual(values = c(`TRUE` = pal[1], `FALSE` = pal_grey)) +
  scale_x_continuous(breaks = pca_var$component) +
  scale_y_continuous(labels = percent) +
  labs(title = "Cumulative variance explained", x = "Component", y = NULL)

scree + cumul
```

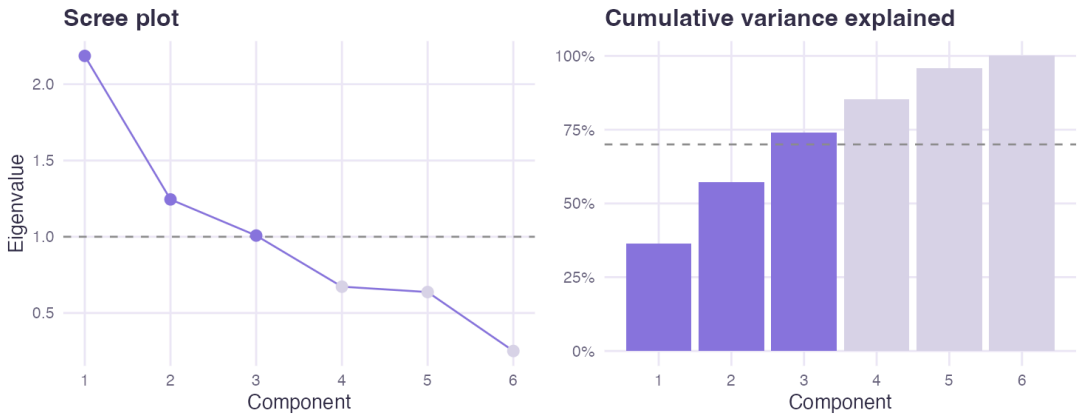


Figure 12: Left: scree plot with the eigenvalue of 1 marked. Right: cumulative variance with the 70% line.

```
eig      <- pca_var$eigenvalue
cum_keep <- pca_var$cumulative[n_keep]
```

Table 30 and Figure 12 show the variance explained. The first component explains 36.4% of the variation on its own, and each further component adds less. Three rules were read together to decide how many to keep:

1. **Kaiser rule** [18] (keep components with an eigenvalue above 1, meaning the component carries more information than a single standardised variable): 3 components.
2. **Share of variance** (keep enough components to explain at least 70% of the variation): 3 components, together explaining 74%. The unit's PCA lecture suggests that more than 60% is enough for social science data such as household purchases [19], so 70% is the stricter of the two and both are met.

3. **Scree plot** (keep the components before the curve flattens): after component 3 the eigenvalues drop to 0.67 and 0.64, which are almost level, so the curve flattens there.

The three rules agree, so 3 components are retained, keeping 74% of the information while reducing six variables to three dimensions. One caution: the third eigenvalue is 1.008, only just above 1, so the Kaiser rule alone would be a weak reason to keep it; the share of variance and the scree plot are what make the case. As a check, base R's `prcomp()` gives exactly the same eigenvalues.

10.4 Naming and describing the components

The **loadings** are the weights each original variable gets in a component. A loading far from 0 (here beyond <U+00B1>0.4, a common cut-off) means the variable helps define that component, and its sign says whether the variable rises or falls as the component score rises. The sign of a whole component is arbitrary in PCA (multiplying every loading by −1 describes the same component), so each component is flipped, where needed, so that its largest loading is positive and "a high score" reads naturally. Figure 13 and Table 31 show the loadings of the retained components.

```
kept_pcs <- paste0("PC", seq_len(n_keep))

pca_load_raw <- tidy(pca_rec, id = "pca", type = "coef") |>
  filter(component %in% kept_pcs) |>
  select(variable = terms, component, value)

# Flip each component so its largest loading is positive (direction only, the
# PCA is unchanged)
pc_flip <- pca_load_raw |>
  group_by(component) |>
  slice_max(abs(value), n = 1, with_ties = FALSE) |>
  ungroup() |>
  transmute(component, flip = sign(value))

pca_load <- pca_load_raw |>
  left_join(pc_flip, by = "component") |>
  mutate(value  = value * flip,
         strong = abs(value) >= 0.4)

pca_load |>
  ggplot(aes(value, fct_rev(factor(variable, levels = cluster_vars)),
             fill = strong)) +
  geom_col() +
  geom_vline(xintercept = c(-0.4, 0.4), linetype = "dashed",
             colour = "grey55") +
  facet_wrap(~ factor(component, levels = kept_pcs)) +
  scale_fill_manual(values = c(`TRUE` = pal[1], `FALSE` = pal_grey),
                    labels = c(`TRUE` = "Defines the component",
                               `FALSE` = "Weak")) +
```

```
  labs(title = "Loadings of the retained components", x = "Loading", y = NULL,
       fill = NULL)
```

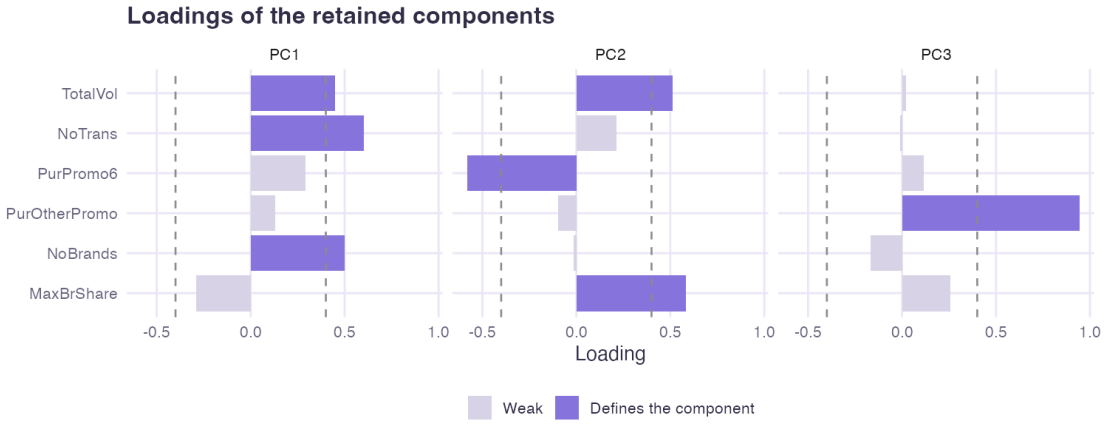


Figure 13: Loadings of the three retained components. Dark bars are beyond plus or minus 0.4 and define the component.

```
pca_load_wide <- pca_load |>
  select(variable, component, value) |>
  pivot_wider(names_from = component, values_from = value) |>
  arrange(factor(variable, levels = cluster_vars))

load_gt <- pca_load_wide |>
  gt() |>
  fmt_number(all_of(kept_pcs), decimals = 2) |>
  cols_label(variable = "Variable")

# Bold the loadings that define each component
for (pc in kept_pcs) {
  load_gt <- load_gt |>
    tab_style(style = cell_text(weight = "bold", color = pal[1]),
              locations = cells_body(columns = all_of(pc),
                                     rows = which(abs(pca_load_wide[[pc]]) >=
                                                    0.4)))
}
load_gt
```

Table 31: Loadings of the retained components (bold: beyond plus or minus 0.4).

Variable	PC1	PC2	PC3
TotalVol	**0.45**	**0.51**	0.02
NoTrans	**0.60**	0.21	–0.01
PurPromo6	0.29	**–0.58**	0.11
PurOtherPromo	0.13	–0.10	**0.95**
NoBrands	**0.50**	–0.01	–0.17
MaxBrShare	–0.29	**0.58**	0.26

```
# Loading of one variable on one component, and a component's share, as text
loading_text <- \(v, pc) {
  sprintf("%+.2f", pca_load_wide[[pc]][pca_load_wide$variable == v])
}
pct_var <- \(pc) {
  round(pca_var$percent[pca_var$component == as.integer(sub("PC", "", pc))],
1)
}
```

PC1: Shopping activity and brand variety (36.4% of the variation). It is defined by NoTrans (+0.60), NoBrands (+0.50) and TotalVol (+0.45), so a high PC1 score means a household that buys often, buys a lot and spreads its purchases across many brands, while a low score is a light, occasional buyer. Because the number of brands loads together with the activity variables, buying more goes with trying more brands: in these data the heaviest buyers are not the most loyal ones (MaxBrShare loads –0.29).

PC2: Loyal bulk buying versus promotion seeking (20.7% of the variation). It is defined by MaxBrShare (+0.58) and TotalVol (+0.51) against PurPromo6 (-0.58). A high PC2 score means a household that puts most of its volume on one brand and buys in quantity without needing a deal; a low score means a household whose purchases lean on promotion 6. This is the loyalty versus deal-seeking axis in the Key Problem, and it is a different idea from how active a household is (PC1). The word "bulk" needs care, because TotalVol also loads +0.45 on PC1. So part of the volume difference between households is already counted as activity in PC1, and because the components are uncorrelated, the volume part of PC2 is the part of volume that PC1 does not already explain.

PC3: Response to other promotions (16.8% of the variation). It is almost entirely PurOtherPromo (+0.95), the variable that was barely related to any other. It earns a component of its own because this kind of promotion response is separate from both activity and loyalty: knowing how much a household buys, or how loyal it is, says little about whether it takes up these offers.

10.5 The households on the components

Each household gets a score on every retained component. Plotting the first two scores shows the households in two dimensions instead of six, coloured by their segment from Question 2, with the loadings drawn as arrows pointing in the direction in which each variable increases.

```
# Component scores, flipped the same way as the loadings, joined to the
segments
pca_scores <- bake(pca_rec, new_data = NULL) |>
  select(MemberID, all_of(kept_pcs)) |>
  mutate(across(all_of(kept_pcs),
                \(x) x * pc_flip$flip[pc_flip$component == cur_column()]))
|>
  left_join(soap_clusters |> select(MemberID, segment), by = "MemberID") |>
  mutate(segment = factor(segment, levels = segment_names))

# Arrows: the PC1 and PC2 loadings, stretched so they are visible next to the
# scores
arrow_scale <- 0.8 * max(abs(c(pca_scores$PC1, pca_scores$PC2)))
pca_arrows  <- pca_load |>
  filter(component %in% c("PC1", "PC2")) |>
  select(variable, component, value) |>
  pivot_wider(names_from = component, values_from = value) |>
  mutate(across(c(PC1, PC2), \(x) x * arrow_scale))

ggplot(pca_scores, aes(PC1, PC2)) +
  geom_point(aes(colour = segment), alpha = 0.9) +
  geom_segment(data = pca_arrows, aes(x = 0, y = 0, xend = PC1, yend = PC2),
               arrow = arrow(length = unit(0.2, "cm")), colour = pal_ink) +
   geom_label(data = pca_arrows, aes(label = variable), size = 3, alpha =
0.8) +
  scale_colour_manual(values = pal) +
  guides(colour = guide_legend(nrow = 2)) +
  labs(title = "Households on the first two components",
       x = "PC1: Shopping activity and brand variety",
       y = "PC2: Loyal bulk buying versus promotion seeking", colour = NULL)
```

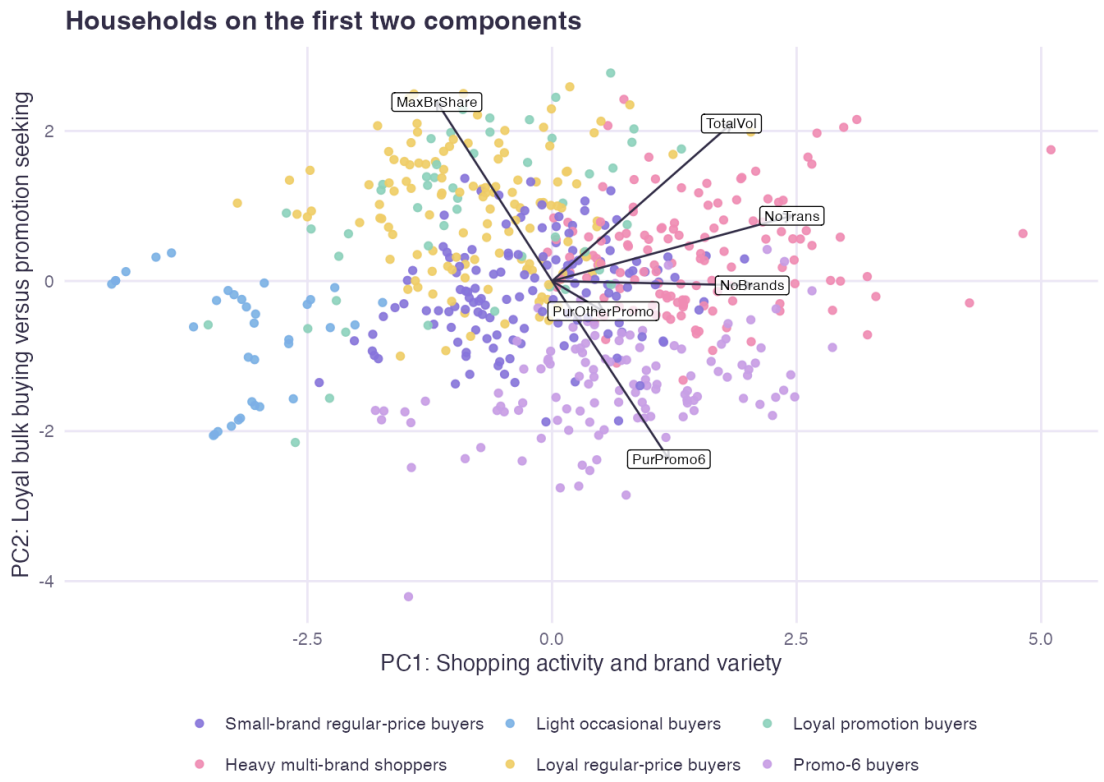


Figure 14: Households on the first two components, coloured by segment. Arrows show the loadings (stretched to be visible; only direction and relative length matter).

```
# Mean score per segment, and R squared of each component's scores on segment
seg_means <- pca_scores |>
  group_by(segment) |>
  summarise(across(all_of(kept_pcs), mean)) |>
  mutate(segment = as.character(segment))

seg_r2 <- map_dbl(kept_pcs, \(pc) summary(lm(reformulate("segment", pc),
                                             data = pca_scores))$r.squared)
|>
  set_names(kept_pcs)

seg_means |>
  bind_rows(tibble(segment = "Variation explained by segment",
                   PC1 = seg_r2[["PC1"]], PC2 = seg_r2[["PC2"]],
                   PC3 = seg_r2[["PC3"]])) |>
  gt() |>
  fmt_number(all_of(kept_pcs),
             rows = segment != "Variation explained by segment",
             decimals = 2) |>
  fmt_percent(all_of(kept_pcs),
              rows = segment == "Variation explained by segment",
```

```
            decimals = 0) |>
  cols_label(segment = "Segment")
```

Table 32: Mean component score of each segment, and the share of each component's variation explained by segment membership.

Segment	PC1	PC2	PC3
Small-brand regular-price buyers	–0.21	–0.15	0.11
Heavy multi-brand shoppers	1.48	0.34	–0.43
Light occasional buyers	–3.18	–0.74	–0.54
Loyal regular-price buyers	–0.75	0.93	–0.40
Loyal promotion buyers	–0.83	0.89	1.98
Promo-6 buyers	0.69	–1.37	0.12
Variation explained by segment	62%	54%	42%

```
seg_score <- \(seg, pc) sprintf("%+.2f",
                                seg_means[[pc]][seg_means$segment == seg])
```

Figure 14 plots the households and Table 32 gives each segment's mean score. The plot reads like a map of the segments. **Moving right** (PC1) goes from the Light occasional buyers (-3.18) to the Heavy multi-brand shoppers (+1.48), the two ends of shopping activity. **Moving up** (PC2) goes from the Promo-6 buyers at the bottom (-1.37) to the Loyal regular-price buyers (+0.93) and Loyal promotion buyers (+0.89) at the top, the loyalty against deal-seeking axis. The MaxBrShare and PurPromo6 arrows point in opposite directions, so households that are loyal to one brand tend not to rely on promotion 6. PC3, not shown in the plot, is where the **Loyal promotion buyers** stand out (+1.98), because they are the segment that takes up other promotions.

Segment membership explains 62% of the variation in PC1, 54% in PC2 and 42% in PC3. **The PCA therefore agrees with the clustering and names the dimensions behind it:** the clustering finds the groups, and the components name the three dimensions on which they differ, which are activity, loyalty against promotion 6, and response to other promotions. This is not independent proof that the segments are real, because both methods use the same six prepared variables.

10.6 What PCA gives up

PCA kept 74% of the information, so 26% of the variation between households is set aside. Three further limits matter for a client:

- **Components are harder to explain than the original variables.** "Shopping activity and brand variety" is a weighted mix of several variables, and a manager may find "number of transactions" easier to act on. This is why the segments in Question 2 were described with the original variables.

- **PCA only finds straight-line relationships, and is pulled by extreme values.** The skew correction and standardisation in Section 7 reduce this, but a curved pattern, or one driven by a small group of heavy promotion users, can be missed or exaggerated.
- **It cannot compress variables that do not overlap.** PurOtherPromo was nearly unrelated to the others, so it needed a component to itself. When most variables are unrelated, PCA saves little; it is most useful when many variables measure a few underlying ideas.

11 Conclusion and Recommendations

11.1 What we found

XYZ asked whether the bath soap market could be segmented by **how households buy** rather than by who they are. It can. Question 1 showed that raw brand shares measure *which* brand a household buys, not *how loyal* it is, so they were replaced by a single concentration measure, `MaxBrShare`. Six behaviour variables covering volume, frequency, promotion response and loyalty were then cleaned, corrected for skewness and standardised (Section 7), and Ward's hierarchical clustering grouped the 600 households into **six segments** (Section 8).

The PCA in Section 10 agrees with the clustering and names the dimensions behind it. It is not independent proof, since both use the same six variables. Three components keep 74% of the information: **shopping activity and brand variety**, **loyal bulk buying against promotion 6**, and **response to other promotions**. Segment membership explains 62%, 54% and 42% of the variation in these three components, so the segments line up with clear, nameable dimensions of behaviour rather than with noise.

The demographic profile in Section 9 adds one important lesson: **the segments are only weakly visible in demographics.** Socioeconomic class and household size differ between segments, but the effects are small, and age does not differ at all. A client could not have found these segments from demographics, which is exactly why XYZ's move to behavioural segmentation is worthwhile.

11.2 Recommendations for XYZ's clients

```
recommendations <- tribble(
  ~segment, ~what_defines_it, ~recommended_approach,
  "Loyal regular-price buyers",
  "Most volume on one brand, almost no promotion buying",
  str_c("Reward and loyalty schemes to keep them; avoid discounts, which ",
        "only give away margin"),
  "Loyal promotion buyers",
  "The most loyal segment, and heavy users of other promotions",
  "Price promotions and value packs on the brand they already favour",
  "Heavy multi-brand shoppers",
  "The most volume and transactions, spread across many brands",
  str_c("Family and multi-packs tied to one brand, with rewards that ",
        "concentrate their spending"),
  "Promo-6 buyers",
  "Low loyalty, heavy use of promotion 6, more higher-class households",
 "Promotion 6 offers spread across the year; a cheaper product is not needed",
```

```
  "Small-brand regular-price buyers",
  str_c("Average volume, little spent on the eight named brands, rarely ",
        "buys on promotion"),
  str_c("Trial and visibility to win them from small brands; discounts are ",
        "unlikely to move them. The least sharply defined segment, so test ",
        "on a small scale first"),
  "Light occasional buyers",
  "Very low volume, few transactions, poorly recorded demographics",
  "Lowest priority for promotion spending"
) |>
  mutate(households = map_int(segment, seg_size), .after = segment)

recommendations |>
  gt() |>
  cols_label(segment = "Segment", households = "Households",
             what_defines_it = "What defines it",
             recommended_approach = "Recommended approach") |>
  cols_width(segment ~ px(170), households ~ px(90))
```

Table 33: One promotion approach per segment, from purchase behaviour (Question 2d) and demographics (Question 2e).

Segment	Households	What defines it	Recommended approach
Loyal regular-price buyers	127	Most volume on one brand, almost no promotion buying	Reward and loyalty schemes to keep them; avoid discounts, which only give away margin
Loyal promotion buyers	49	The most loyal segment, and heavy users of other promotions	Price promotions and value packs on the brand they already favour
Heavy multi-brand shoppers	132	The most volume and transactions, spread across many brands	Family and multi-packs tied to one brand, with rewards that concentrate their spending
Promo-6 buyers	117	Low loyalty, heavy use of promotion 6, more higher-class households	Promotion 6 offers spread across the year; a cheaper product is not needed
Small-brand regular-price buyers	140	Average volume, little spent on the eight named brands, rarely buys on promotion	Trial and visibility to win them from small brands; discounts are unlikely to move them. The least sharply defined segment, so test on a small scale first
Light occasional buyers	35	Very low volume, few transactions, poorly recorded demographics	Lowest priority for promotion spending

The segments differ in *when* and *how* a promotion pays off, which is what lets a client spread several promotions across the year instead of one blanket discount. Table 33 sets out

one approach per segment. The two loyal segments together hold 176 households, and they need opposite treatment: one should be rewarded without discounts, the other responds to discounts on its own brand. Treating them as one "loyal" group, as a demographic segmentation would, would waste the promotion budget on the first and under-serve the second.

11.3 Limitations

- **The segments overlap.** The average silhouette width at six clusters is only 0.158, and the adjusted Rand index against k-means is 0.389 (Section 8.3.b). The overlap is uneven: the average silhouette width is 0.467 for the Light occasional buyers but only 0.073 for the Small-brand regular-price buyers, so households near the borders of the weaker segments could reasonably belong to a neighbouring one.
- **Six groups is partly a judgement.** The silhouette slightly favours two groups; six was chosen because two would mix loyal and light buyers, which no client could act on (Section 8.2).
- **Data quality.** Demographics coded 0 were treated as missing, and `CHILD` = 5 was treated as an undocumented "not specified" code. 2 households have brand shares that do not sum to 1, and the largest share of the 68 households with no household size (and no education, language, gender, TV or Affluence) sits in the Light occasional segment.
- **Skewness in the promotion shares remains.** About half the households never use each promotion, so even after the square root the promotion shares stay skewed. This reflects real behaviour but makes those variables less symmetric than the others.
- **One sample, one period, no causation.** The analysis uses a subset of 600 households over a single period. The recommendations describe which promotion *fits* each segment's behaviour; they do not prove that a promotion *causes* a change, which only a controlled test could show.

11.4 Next steps

1. Run the same segmentation on the full panel and on a later period, to check that the six segments are stable and to see which households move between them.
2. Test the recommended promotions with a controlled trial in one market before rolling them out, so their effect on each segment can be measured.
3. Consider a brand switching measure that is not built from `NoTrans`, which Section 6 showed is the problem with `BrandRuns` and `TransPerRun`, to capture the third view of loyalty in the brief.

Appendix

Further checks

These checks support the main text but are not needed to follow it, so they sit here.

Question 1(b): a second method gives the same MaxBrShare

```
# pmax() works across whole columns at once instead of one row at a time
max_pmax <- do.call(pmax, select(soap_raw, all_of(brand_cols)))
# Stops the render if the two methods ever disagree
stopifnot(isTRUE(all.equal(soap$MaxBrShare, max_pmax)))
n_same <- sum(near(soap$MaxBrShare, max_pmax))
```

The vectorised `pmax()` gives the same value for all 600 households (the `stopifnot()` check would stop the render otherwise). It is the more efficient form because it avoids creating a group for each row. The `rowwise()` approach is kept above because it shows the row-by-row logic most clearly.

Question 1(b): how concentration is spread

```
p_hist <- ggplot(soap, aes(MaxBrShare)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = pal[1],
                 colour = "white") +
  scale_x_continuous(labels = percent) +
  labs(title = "Concentration on one named brand", x = "MaxBrShare",
       y = "Households")

p_scatter <- ggplot(soap, aes(NoBrands, MaxBrShare)) +
  geom_point(colour = pal[1], alpha = 0.7) +
   stat_summary(fun = median, geom = "line", colour = pal[2], linewidth =
0.9) +
  scale_y_continuous(labels = percent) +
   labs(title = "More brands, less concentration", x = "Number of brands
bought",
       y = "MaxBrShare")

p_hist + p_scatter
```

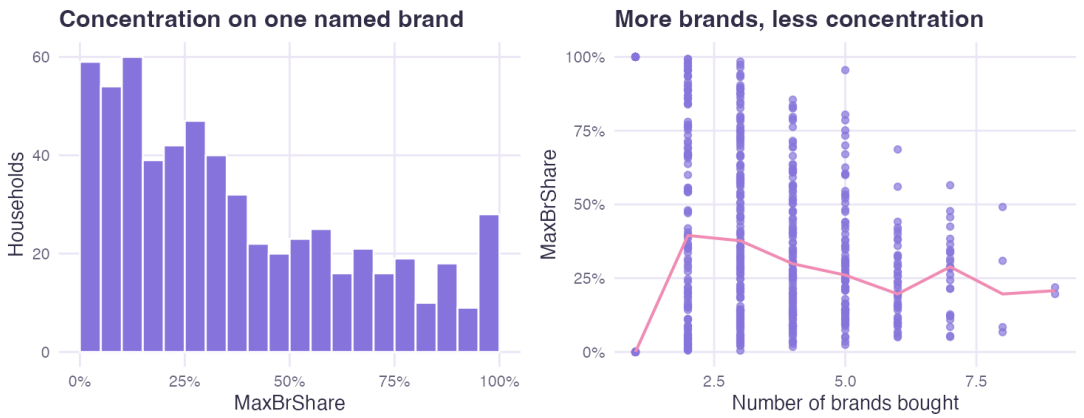


Figure 15: Left: distribution of MaxBrShare. Right: MaxBrShare against the number of brands bought (line = median at each brand count).

Figure 15 shows the spread. The middle half of households give between 13% and 57% of their volume to their favourite named brand, and 15 give it all of their volume. Concentration falls as the number of brands rises, but households with the same brand count still differ widely, and the single-brand households include both 100% named-brand buyers and the zero group. This is why the two variables are read together.

Question 1(b): do the nine shares add up to 1?

598 households have nine shares (eight named brands plus `BrOthers`) that sum to 1 within 0.01. The 2 exceptions are shown in Table 34. They are kept as supplied, because rescaling them would change the measure the question defines. Their `MaxBrShare` values are still valid shares between 0 and 1.

```
share_exceptions |>
  gt() |>
  fmt_number(share_total, decimals = 3) |>
 cols_label(MemberID = "Household ID", share_total = "Sum of the nine shares")
```

Table 34: Households whose nine brand shares do not sum to 1 (share_exceptions from Section 3).

Household ID	Sum of the nine shares
1061080	0.934
1061130	1.022

Question 2(e): where do the differences come from?

The tests above say *whether* the segments differ. This check says *which* segment and class cause it, so a client knows where to act. For SEC and AGE, the **standardised residual** of each cell compares the number of households in that segment and class with the number expected if every segment had the same mix. A value beyond <U+00B1>2 marks a clear excess (positive) or shortfall (negative). For HS, **pairwise Wilcoxon tests** compare every pair of segments, with the **Holm adjustment** keeping the overall chance of a false difference at 5% across all the comparisons.

```
# Standardised residuals of the segment by class table, in long form for
# plotting
resid_df <- function(data, var) {
  d   <- data |> filter(!is.na(.data[[var]]))
  tab <- table(segment = d$segment, level = d[[var]])
  # Simulated p-value, so no small-cell warning. Only the residuals are used
  res <- chisq.test(tab, simulate.p.value = TRUE, B = 2000)
  as.data.frame.table(res$stdres, responseName = "resid") |>
    mutate(variable = var)
}

plot_resid <- function(rd, title) {
  ggplot(rd, aes(level, fct_rev(segment), fill = resid)) +
    geom_tile(colour = "white") +
    geom_text(aes(label = round(resid, 1), colour = abs(resid) > 2),
              show.legend = FALSE) +
    scale_colour_manual(values = c(`TRUE` = pal_ink, `FALSE` = pal_ink)) +
    scale_fill_gradient2(low = pal_lo, mid = "white", high = pal_hi,
                         midpoint = 0) +
```

```
    labs(title = title, x = NULL, y = NULL, fill = "Residual")
}

sec_resid <- resid_df(profile_df, "SEC")
age_resid <- resid_df(profile_df, "AGE")

plot_resid(sec_resid, "Socioeconomic class") +
  plot_resid(age_resid, "Age of the homemaker")
```

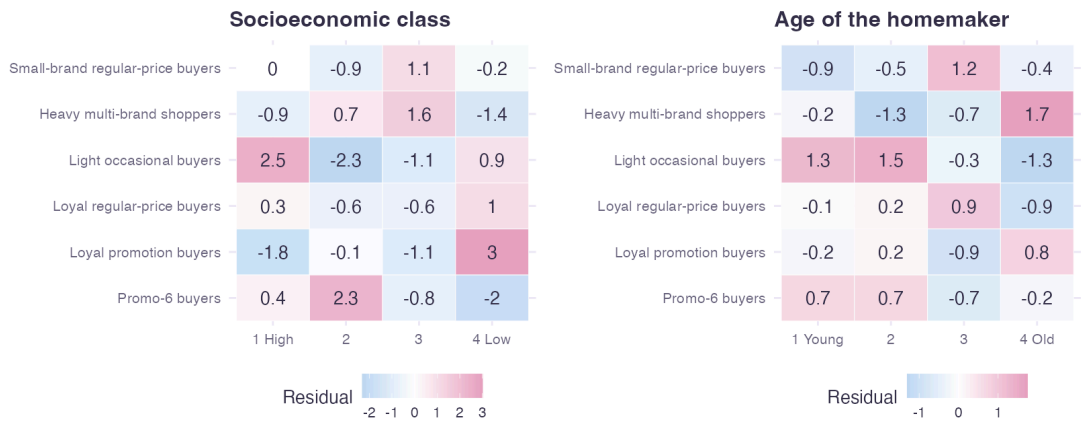


Figure 16: Standardised residuals of segment by SEC and by AGE. Beyond plus or minus 2 is a clear difference.

```
hs_pairs <- pairwise.wilcox.test(hs_df$HS, hs_df$segment,
                                 p.adjust.method = "holm", exact = FALSE) |>
  tidy()

hs_pairs |>
  filter(p.value < 0.05) |>
  gt() |>
  fmt_number(p.value, decimals = 4) |>
  cols_label(group1 = "Segment", group2 = "Compared with",
             p.value = "Holm-adjusted p-value")
```

Table 35: Household size: segment pairs that still differ after the Holm adjustment.

Segment	Compared with	Holm-adjusted p-value
Heavy multi-brand shoppers	Small-brand regular-price buyers	0.0165
Promo-6 buyers	Heavy multi-brand shoppers	0.0011

```
resid_text <- \(seg, lvl) {
  sprintf("%+.1f",
          sec_resid$resid[sec_resid$segment == seg & sec_resid$level == lvl])
}
n_age_cells <- sum(abs(age_resid$resid) > 2)
```

Figure 16 and Table 35 show the detail. The SEC result is driven by three segments: the Loyal promotion buyers have more SEC 4 households than expected (+3.0), the Promo-6 buyers have more SEC 2 households (+2.3), and the small Light occasional group has more SEC 1 households (+2.5). No age cell goes beyond 2 in either direction, which matches the non-significant age test. For household size, only the pairs in the table above differ after adjustment, and every one of them involves the Heavy multi-brand shoppers' larger households. These are the comparisons a client should rely on when choosing which segment gets which offer.

Question 2(e): did we miss a better demographic?

The three variables were chosen for business reasons. As a check, every demographic is measured on how strongly it separates the segments with one common measure, Cram<U+00E9>r's V. Numeric variables (HS, Affluence and EDU, which has nine sparse levels) are first cut into four equal-sized groups, and native-language codes held by fewer than 20 households are pooled, so that rare codes do not inflate the measure.

```
all_demo_v <- map_dfr(demo_cols, \(v) {
  d <- soap_clusters |>
    select(MemberID, segment) |>
    left_join(soap_clean |> select(MemberID, value = all_of(v)),
              by = "MemberID") |>
    filter(!is.na(value))
  grp <- if (v %in% c("HS", "Affluence", "EDU")) {
    # numeric: four equal-sized groups
    factor(ntile(d$value, 4))
  } else {
    # categorical: pool rare codes
    fct_lump_min(factor(d$value), min = 20)
  }
  tab <- table(d$segment, grp)
  # Simulated p-value, so no small-cell warning. Only the statistic is used
  res <- chisq.test(tab, simulate.p.value = TRUE, B = 2000)
  tibble(variable = v, n = nrow(d),
         cramers_v = cramers_v(tab, unname(res$statistic)),
         chosen = v %in% profile_vars)
})

all_demo_v |>
  ggplot(aes(cramers_v, fct_reorder(variable, cramers_v), fill = chosen)) +
  geom_col() +
  scale_fill_manual(values = c(`TRUE` = pal[1], `FALSE` = pal_grey),
```

```
                    labels = c(`TRUE` = "Chosen", `FALSE` = "Not chosen")) +
  labs(title = "How strongly each demographic separates the segments",
       x = "Cramer's V", y = NULL, fill = NULL)
```

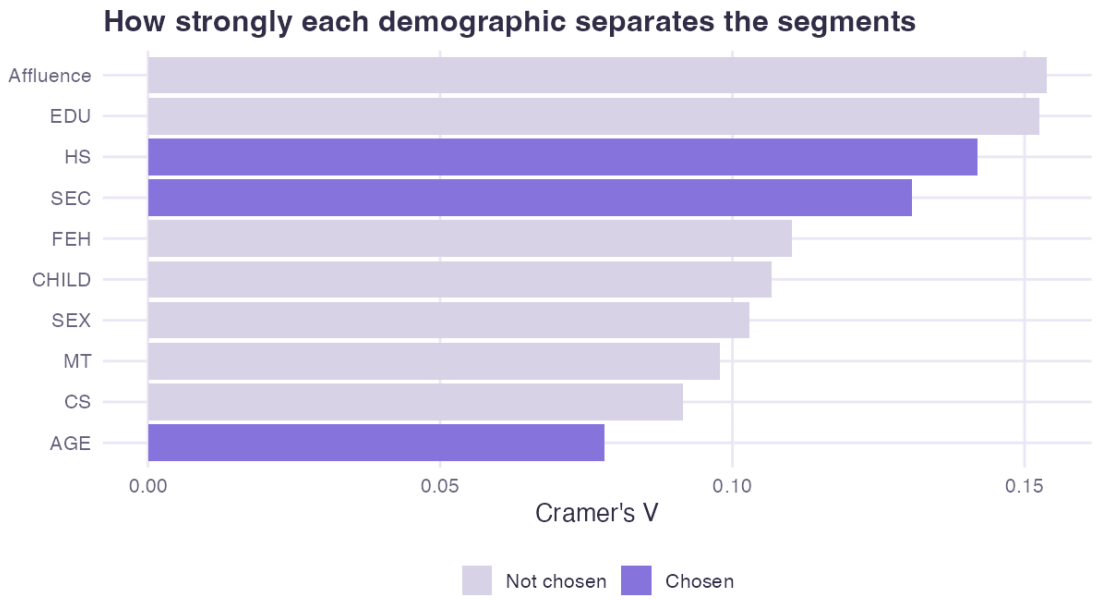


Figure 17: How strongly each demographic separates the six segments (Cramer's V).

```
demo_v <- \(v) round(all_demo_v$cramers_v[all_demo_v$variable == v], 3)
```

Figure 17 shows that every demographic separates the segments only weakly (the largest V is 0.154), which repeats the finding above. HS and SEC are among the stronger variables, which supports two of the three choices. `Affluence` (V = 0.154) and `EDU` (0.153) separate the segments more than AGE (0.078). They were left out because Affluence largely repeats SEC and EDU has many sparse levels and more missing values. Since the brief lets the team choose any three demographics, Affluence would be an acceptable alternative to AGE. It is reported here so a reader can see the trade-off rather than take the choice on trust.

Packages used

Table 36 records the installed versions of every package explicitly loaded in the shared setup, together with the rendering dependencies. These versions are read from the R library at execution time, rather than transcribed from a previous run. The complete attached-package and namespace information follows in `sessionInfo()`.

```
# Keep this list aligned with the explicit package calls in the shared setup.
appendix_packages <- tibble(
  package = c("pacman", "tidyverse", "tidymodels", "here", "janitor", "gt",
              "scales", "patchwork", "factoextra", "cluster", "moments",
              "mclust",
              "knitr", "rmarkdown", "ragg"),
  role = c(rep("Shared setup", 12), rep("Rendering", 3))
```

```
) |>
  mutate(version = purrr::map_chr(package,
                                  \(pkg) as.character(packageVersion(pkg))))

appendix_packages |>
  select(package, version, role) |>
  gt() |>
  cols_label(package = "Package", version = "Installed version",
             role = "Use in this report")
```

Table 36: R packages loaded by the shared setup and dependencies used for rendering.

Package	Installed version	Use in this report
pacman	0.5.1	Shared setup
tidyverse	2.0.0	Shared setup
tidymodels	1.5.0	Shared setup
here	1.0.2	Shared setup
janitor	2.2.1	Shared setup
gt	1.3.0	Shared setup
scales	1.4.0	Shared setup
patchwork	1.3.2	Shared setup
factoextra	2.2.0	Shared setup
cluster	2.1.8.2	Shared setup
moments	0.14.1	Shared setup
mclust	6.1.3	Shared setup
knitr	1.51	Rendering
rmarkdown	2.31	Rendering
ragg	1.5.2	Rendering

The table identifies the computational environment used for this particular render. Matching these versions helps another reader investigate differences between runs, although a version table alone does not install or lock dependencies. R and Quarto versions are recorded in Section 2.

Session information

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
```

```
Running under: macOS Tahoe 26.6.2

Matrix products: default
BLAS:           /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/
libRblas.0.dylib
LAPACK:         /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/
libRlapack.dylib;  LAPACK version 3.12.1

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Asia/Kuala_Lumpur
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] moments_0.14.1    cluster_2.1.8.2   factoextra_2.2.0  patchwork_1.3.2
 [5] gt_1.3.0          janitor_2.2.1     here_1.0.2        yardstick_1.4.0
 [9] workflowsets_1.1.1 workflows_1.3.0   tune_2.1.0        tailor_0.1.0
[13] rsample_1.3.2     recipes_1.4.0     parsnip_1.6.0     modeldata_1.6.0
[17] infer_1.1.0       dials_1.4.4       scales_1.4.0      broom_1.0.13
[21] tidymodels_1.5.0  lubridate_1.9.5   forcats_1.0.1     stringr_1.6.0
[25] dplyr_1.2.1       purrr_1.2.2       readr_2.2.0       tidyr_1.3.2
[29] tibble_3.3.1      ggplot2_4.0.3     tidyverse_2.0.0   pacman_0.5.1

loaded via a namespace (and not attached):
 [1] gridExtra_2.3.1     rlang_1.2.0         magrittr_2.0.5
 [4] snakecase_0.11.1    furrr_0.4.0         otel_0.2.0
 [7] compiler_4.6.0      systemfonts_1.3.2   vctrs_0.7.3
[10] crayon_1.5.3        pkgconfig_2.0.3     fastmap_1.2.0
[13] backports_1.5.1     labeling_0.4.3      rmarkdown_2.31
[16] prodlim_2026.03.11  tzdb_0.5.0          ragg_1.5.2
[19] bit_4.6.0           xfun_0.57           jsonlite_2.0.0
[22] parallel_4.6.0      R6_2.6.1            stringi_1.8.7
[25] RColorBrewer_1.1-3  car_3.1-5           parallelly_1.48.0
[28] rpart_4.1.27        Rcpp_1.1.1-1.1      knitr_1.51
[31] future.apply_1.20.2 Matrix_1.7-5        splines_4.6.0
[34] nnet_7.3-20         timechange_0.4.0    tidyselect_1.2.1
[37] viridis_0.6.5       abind_1.4-8         rstudioapi_0.18.0
[40] yaml_2.3.12         timeDate_4052.112   codetools_0.2-20
[43] listenv_1.0.0       lattice_0.22-9      withr_3.0.2
[46] S7_0.2.2            evaluate_1.0.5      future_1.76.0
[49] survival_3.8-6      xml2_1.5.2          mclust_6.1.3
[52] ggpubr_1.0.0        pillar_1.11.1       carData_3.0-6
[55] generics_0.1.4      vroom_1.7.1         rprojroot_2.1.1
```

```
[58] hms_1.1.4           globals_0.19.1      class_7.3-23
[61] glue_1.8.1          tools_4.6.0         dendextend_1.19.1
[64] data.table_1.18.4   ggsignif_0.6.4      gower_1.0.2
[67] fs_2.1.0            grid_4.6.0          ipred_0.9-16
[70] Formula_1.2-6       cli_3.6.6           DiceDesign_1.10
[73] textshaping_1.0.5   viridisLite_0.4.3   lava_1.9.3
[76] gtable_0.3.6        rstatix_1.1.0       sass_0.4.10
[79] digest_0.6.39       ggrepel_0.9.8       farver_2.1.2
[82] htmltools_0.5.9     lifecycle_1.0.5     hardhat_1.4.3
[85] sparsevctrs_0.3.6   bit64_4.8.2         MASS_7.3-65
```

This output records the running R version, operating system, locale, attached packages and loaded namespaces. It supplements the explicit package table with dependencies loaded indirectly by the analysis.

Bibliography

[1] R Core Team, “R: A Language and Environment for Statistical Computing.” Vienna, Austria, 2026. [Online]. Available: https://www.r-project.org/

[2] H. Wickham, M. Averick, J. Bryan, W. Chang, L. D. McGowan, and others, “Welcome to the tidyverse,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1686, 2019, doi: 10.21105/joss.01686.

[3] M. Kuhn and H. Wickham, “Tidymodels: a collection of packages for modeling and machine learning using tidyverse principles.” 2020. [Online]. Available: https://www.tidymodels.org/

[4] M. Kuhn, H. Wickham, and E. Hvitfeldt, “recipes: Preprocessing and Feature Engineering Steps for Modeling.” 2026. [Online]. Available: https://github.com/tidymodels/recipes

[5] M. Maechler, P. Rousseeuw, A. Struyf, and M. Hubert, “cluster: "Finding Groups in Data": Cluster Analysis Extended Rousseeuw et al..” 2026. [Online]. Available: https://svn.r-project.org/R-packages/trunk/cluster/

[6] A. Kassambara and F. Mundt, “factoextra: Extract and Visualize the Results of Multivariate Data Analyses.” 2026. [Online]. Available: https://github.com/kassambara/factoextra

[7] R. Iannone *et al.*, “gt: Easily Create Presentation-Ready Display Tables.” 2026. [Online]. Available: https://gt.rstudio.com/

[8] K. Müller, “here: A Simpler Way to Find Your Files.” 2025. [Online]. Available: https://here.r-lib.org/

[9] S. Firke, “janitor: Simple Tools for Examining and Cleaning Dirty Data.” 2024. [Online]. Available: https://github.com/sfirke/janitor

[10] T. L. Pedersen, “patchwork: The Composer of Plots.” 2025. [Online]. Available: https://patchwork.data-imaginist.com/

[11] L. Scrucca, M. Fop, T. B. Murphy, and A. E. Raftery, "mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models," *The R Journal*, vol. 8, no. 1, pp. 289–317, 2016, doi: 10.32614/RJ-2016-021.

[12] L. Komsta and F. Novomestky, "moments: Moments, Cumulants, Skewness, Kurtosis and Related Tests." 2022. doi: 10.32614/CRAN.package.moments.

[13] I.-K. Yeo and R. A. Johnson, "A new family of power transformations to improve normality or symmetry," *Biometrika*, vol. 87, no. 4, pp. 954–959, 2000, doi: 10.1093/biomet/87.4.954.

[14] J. H. Ward, "Hierarchical Grouping to Optimize an Objective Function," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963, doi: 10.1080/01621459.1963.10500845.

[15] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[16] L. Hubert and P. Arabie, "Comparing Partitions," *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985, doi: 10.1007/BF01908075.

[17] H. Cramér, *Mathematical Methods of Statistics*. Princeton: Princeton University Press, 1946. doi: 10.1515/9781400883868.

[18] H. F. Kaiser, "The Application of Electronic Computers to Factor Analysis," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 141–151, 1960, doi: 10.1177/001316446002000116.

[19] ETW2500 Teaching Team, "Principal Component Analysis (PCA)." 2026.